

Лабораторная работа №10-11. Создание приложения «Список студентов» с использованием Material Design

Цель работы: Научиться применять принципы Material Design в Android-приложениях с использованием Jetpack Compose. В результате лабораторной работы будет создано приложение «Список студентов», которое демонстрирует использование цветовых схем, типографики, форм и анимаций Material Design.

Теоретическая часть

Что такое Material Design?

Material Design — это система дизайна, разработанная Google для создания качественных цифровых продуктов для Android, веб и других платформ. Она предоставляет рекомендации по созданию читабельного, привлекательного и согласованного пользовательского интерфейса.

Material Design основан на принципах:

- **Материальность** — интерфейс ведёт себя как физические объекты
- **Смелость, графичность, осмысленность** — использование цвета, типографики и пространства
- **Движение** — анимации и переходы создают понятность

Основные компоненты Material Theming

1. Color (Цвет)

Material Design использует **цветовую схему** (Color Scheme), которая определяет палитру цветов для всего приложения:

- **Primary** — основной цвет приложения (кнопки, основные элементы)
- **Secondary** — второстепенный цвет (акценты, FAB)
- **Tertiary** — третичный цвет (дополнительные акценты)
- **Surface** — цвет поверхностей (карточки, листы)
- **Background** — фоновый цвет
- **Error** — цвет для ошибок

Каждый цвет имеет варианты:

- **Container** — контейнерный цвет
- **On[Color]** — цвет текста/иконок на этом цвете

Light Scheme

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

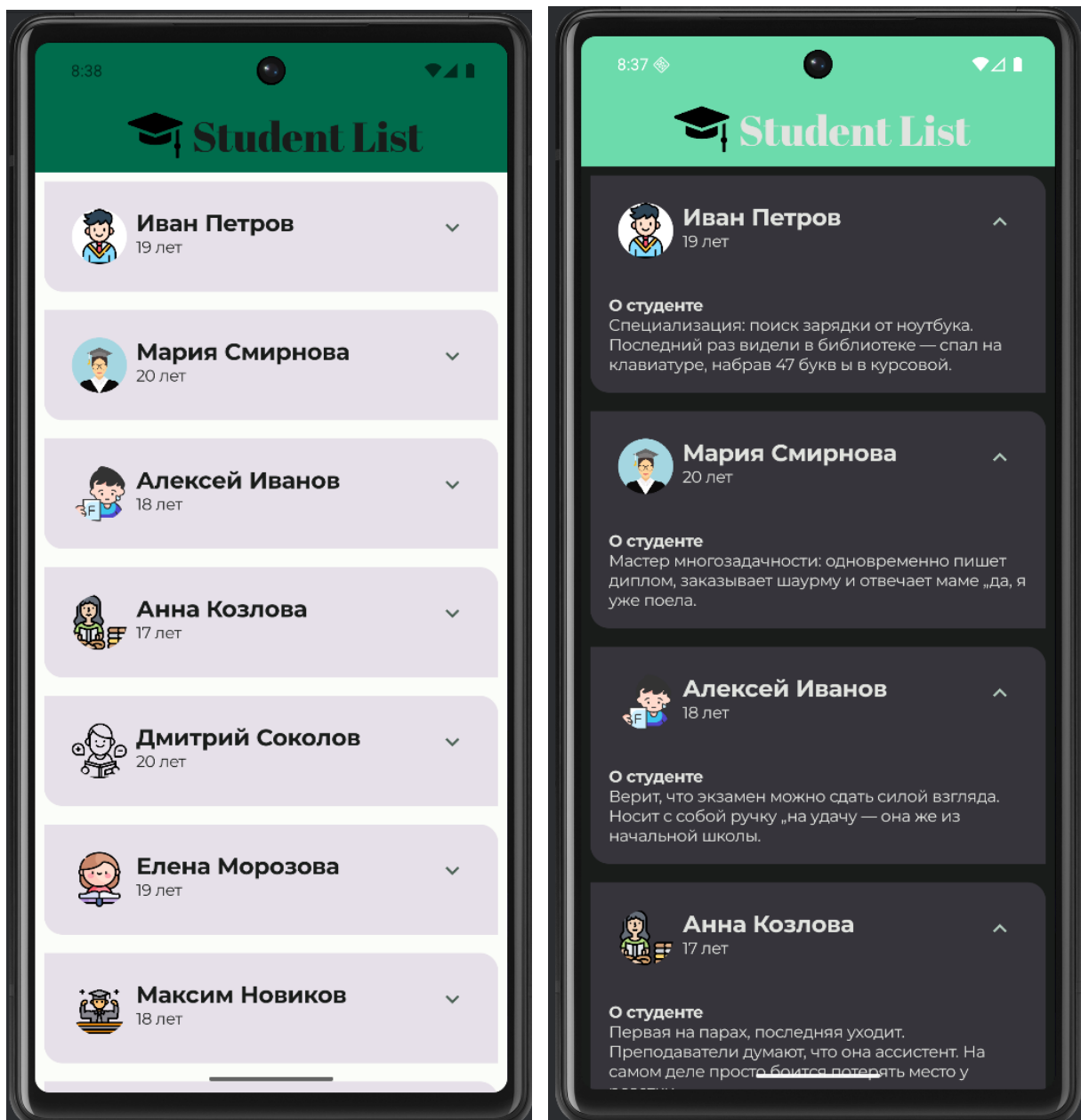
Dark Scheme

Primary	On Primary	Primary Container	On Primary Container
Secondary	On Secondary	Secondary Container	On Secondary Container
Tertiary	On Tertiary	Tertiary Container	On Tertiary Container
Error	On Error	Error Container	On Error Container
Background	On Background	Surface	On Surface
Outline		Surface-Variant	On Surface-Variant

Поддержка тем:

- **Light Theme** (светлая тема)
- **Dark Theme** (тёмная тема)
- **Dynamic Color** (динамические цвета на Android 12+)

Приложение которое мы **создадим** в рамках лабораторных работ:



2. Typography (Типографика)

Material Design определяет набор текстовых стилей:

Type Role	Font	Font weight	Size
displayLarge	Abril Fatface	Normal	36sp
displayMedium	Montserrat	Bold	20sp
labelSmall	Montserrat	Bold	14sp
bodyLarge	Montserrat	Normal	14sp

Каждый стиль включает:

- **fontSize** — размер шрифта
- **lineHeight** — высота строки
- **fontWeight** — толщина шрифта
- **fontFamily** — семейство шрифтов

3. Shape (Форма)

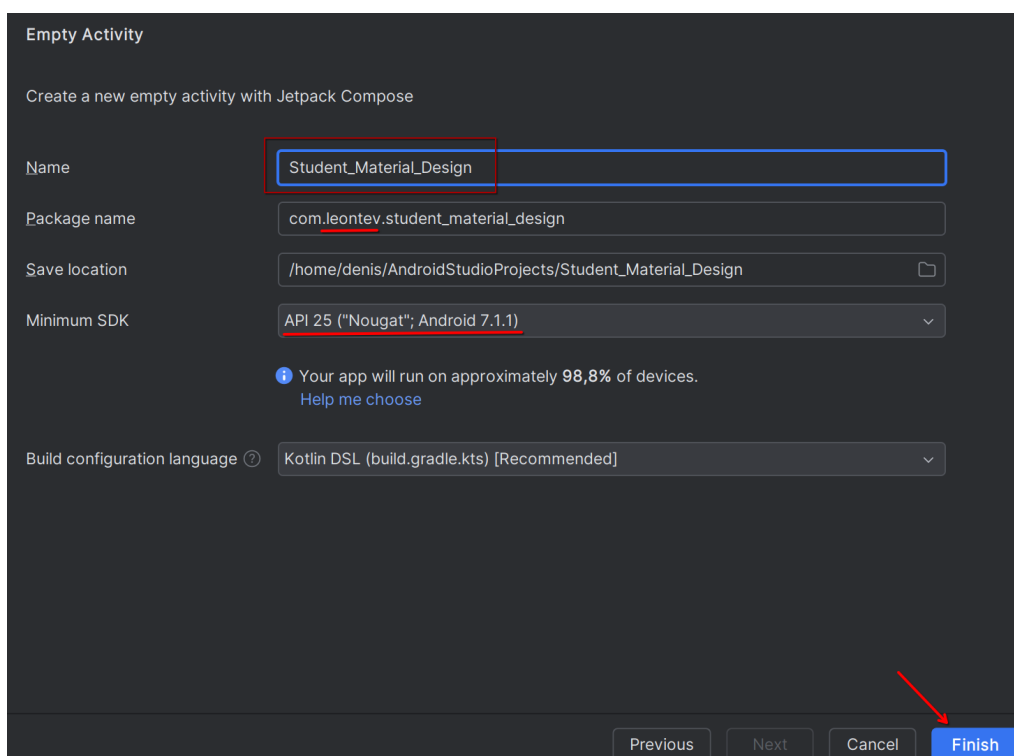
Material Design использует **скругления углов** для создания визуальной иерархии:

- **Small** — 4dp — чипы, маленькие кнопки
- **Medium** — 8dp — карточки, кнопки
- **Large** — 16dp — большие карточки, листы

Практическая часть

Шаг 1. Создание проекта

1. Откройте **Android Studio**
2. Выберите **File → New → New Project**
3. В окне выбора шаблона выберите **Empty Activity → Next**
4. Укажите параметры:
 - **Name:** Student_Material_Design
 - **Minimum SDK:** API 25
5. Нажмите **Finish**



Дождитесь полной загрузки проекта.

Откройте MainActivity.kt и удалите лишние функции, оставив базовую структуру:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContentView {
            Student_Material_DesignTheme {

            }
        }
    }
}
```

2. Создание проекта

Создание Git-репозитория

Перед началом работы создайте Git-репозиторий для отслеживания изменений и защиты от потери данных.

Вариант 1: Через Android Studio (рекомендуется)

1. В Android Studio выберите **VCS → Enable Version Control Integration**
2. Выберите **Git → OK**
3. Выберите **Git → Commit** (или **Ctrl+K**)
4. Выберите все файлы, введите сообщение: **"Initial commit"**
5. Нажмите **Commit**

Вариант 2: Через терминал

1. Откройте терминал в Android Studio (**View → Tool Windows → Terminal**)
2. Выполните команды:

git init

git add .

git commit -m "Initial commit"

Создание удалённого репозитория на GitHub

1. Откройте **GitHub** и войдите в аккаунт.
2. Нажмите **New repository** (зелёная кнопка)
3. **Repository name:** **Student_Material_Design_FIO** (где **FIO** — ваша фамилия)
4. Выберите **Public** (публичный репозиторий)
5. **НЕ** ставьте галочки на "Add README" и ".gitignore"
6. Нажмите **Create repository**

Привязка локального репозитория к удалённому

В терминале Android Studio выполните:

git branch -M main

git remote add origin <URL_репозитория>

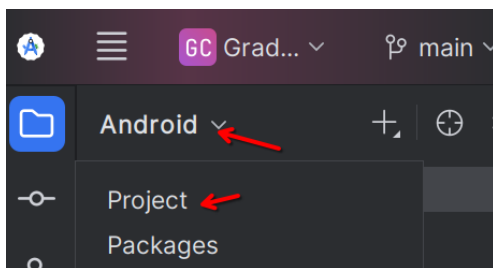
git push -u origin main

* Замените <URL_репозитория> на скопированный URL

При первом **push** вас попросят ввести логин и пароль **GitHub**.

3. Создание папки **img**

1. В Android Studio переключитесь в режим просмотра **Project** (в левом верхнем углу панели Project)



2. Найдите **app**

3. Кликните правой кнопкой на папке **app**

4. Выберите **New** → **Directory**

5. Введите имя: **img**

6. Нажмите **OK**

Полный путь к папке: /app/img/

Правила именования скриншотов

Все скриншоты должны называться по формату:

step[номер]_[описание]_[тема]_FIO.png

Примеры:

- step5_app_running_lightTheme_Ivanov.png

Где:

- step[номер] — номер шага лабораторной работы
- [описание] — краткое описание того, что на скриншоте
- [тема] — lightTheme или darkTheme
- FIO — **ваша фамилия** (например, Ivanov, Petrov, Smirnova)

Шаг 2. Подготовка ресурсов

2.1. Добавление изображений студентов

1. Скачайте или подготовьте **10 изображений** студентов
2. Переименуйте их: student1.jpg, student2.jpg, ..., student10.jpg
3. Перетащите все изображения в папку **res/drawable/**

```
s/AndroidStudioProjects/Student_Material_Design/app/src/main/res/drawable/
```

Рекомендации:

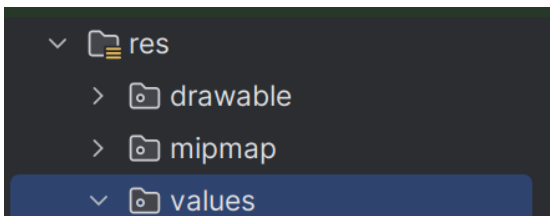
- Размер: минимум 300x300px
- Формат: JPG или PNG
- Желательно квадратные фотографии (1:1)

2.2. Создание файла **dimens.xml**

Файл **dimens.xml** используется для хранения размерных значений (отступы, размеры элементов), что позволяет легко изменять их в одном месте для всего приложения.

Создайте файл **res/values/dimens.xml** для хранения размеров:

1. Кликните правой кнопкой на **res/values**



2. **New** → **Values Resource File**
3. Имя файла: **dimens**
4. Нажмите **OK**

Добавьте следующий код:

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Padding -->
    <dimen name="padding_small">8dp</dimen>
    <dimen name="padding_medium">16dp</dimen>
    <!-- Image size -->
    <dimen name="image_size">64dp</dimen>
</resources>
```

- **padding_small** — маленькие отступы между элементами
- **padding_medium** — средние отступы
- **image_size** — размер аватарки студента (64x64dp)

2.3. Настройка strings.xml

Откройте **res/values/strings.xml**.

Добавьте строки:

```
<resources>
    <string name="app_name">Student List</string>
    <!-- Student names -->
    <string name="student1">Иван Петров</string>
</resources>
```

Далее добавьте ещё **9 студентов**. Примеры имён для вставки:

2. `<string name="student2">Мария Смирнова</string>`

3. Алексей Иванов

4. Анна Козлова

5. Роман Соколов

6. Елена Морозова

7. Максим Новиков

8. Ольга Волкова

9. Сергей Лебедев

10. Татьяна Егорова

Далее сюда же добавим **описание для студентов**. Например:

```
<resources>
    <string name="app_name">Student List</string>
    <!-- Student names -->
    ...
    <string name="student1_desc">Специализация: поиск зарядки от ноутбука. Последний раз
        видели в библиотеке — спал на клавиатуре, набрав 47 букв "ы" в курсовой.</string>
</resources>
```

Примерный текст для вставки. Можете придумать свой:

1. Специализация: поиск зарядки от ноутбука. Последний раз видели в библиотеке — спал на клавиатуре, набрав 47 букв «ы» в курсовой.

2. `<string name="student2_desc">Мастер многозадачности: одновременно пишет диплом, заказывает шаурму и отвечает маме „да, я уже поела".</string>`

3. Верит, что экзамен можно сдать силой взгляда. Носит с собой ручку „на удачу” — она же из начальной школы.

4. Первая на парах, последняя уходит. Преподаватели думают, что она ассистент. На самом деле просто боится потерять место у розетки.

5. Специалист по „я это сделаю завтра”. Календарь забит напоминаниями „срочно”, все даты — вчера. Любит делать проекты по Kotlin на Java.

6. Единственный человек, который приносит на пары настоящий термос. В нём — не чай, а надежда на тройку.

7. Знает все мемы про программирование, но Hello World запускал с третьего раза. Гордость: выжил после дедлайна по ночам.

8. Ведёт 7 тетрадей разных цветов. На экзамене путает их местами и получает пятёрку по химии вместо истории.

9. Специалист по „я болел”. Пропустил 12 пар, но на зачёте знает больше всех — смотрел лекции на 2x скорости в метро.

10. Может объяснить теорию относительности, но не может объяснить, почему в холодильнике до сих пор лежит йогурт с прошлого семестра.

Добавляем **years_old** для отображения возраста:

```
<resources>
  <string name="app_name">Student List</string>
  <!-- Student names -->
  ...
  <!-- Student descriptions -->
  ...
  <!-- Age format -->
  <string name="years_old">%d лет</string>
</resources>
```

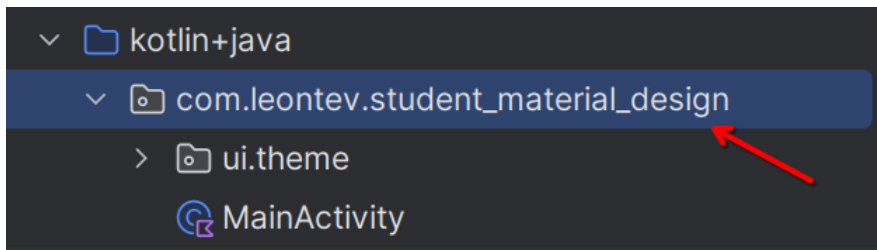
%d — это плейсхолдер, который заменится на число (возраст студента).

Шаг 3. Создание модели данных

3.1. Создание пакета data

Пакет data будет содержать классы для работы с данными приложения.

1. Кликните правой кнопкой на **com.example.student_material_design**

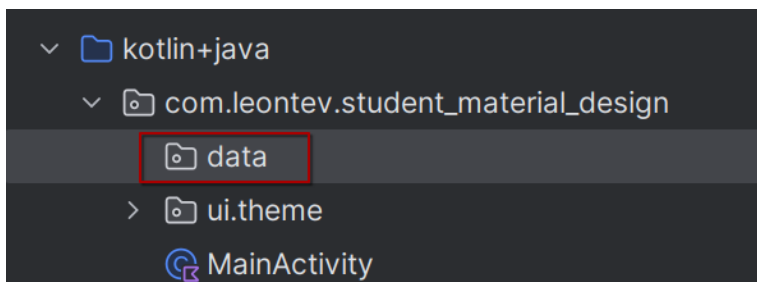


2. **New** → **Package**

3. Введите: **data**

`com.leontev.student_material_design.data`

Итог:



3.2. Создание data class Student

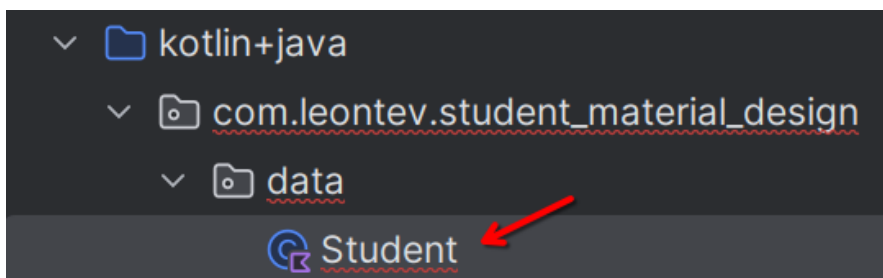
Data class — это специальный класс Kotlin для хранения данных. Он автоматически создаёт методы `equals()`, `hashCode()`, `toString()`.

1. Кликните правой кнопкой на пакете **data**

2. **New** → **Kotlin Class/File**

3. Выберите **Class**

4. Имя: **Student**



Добавьте код:

```
data class Student(  
    @DrawableRes val imageResourceId: Int,  
    @StringRes val name: Int,  
    val age: Int,  
    @StringRes val description: Int  
)
```

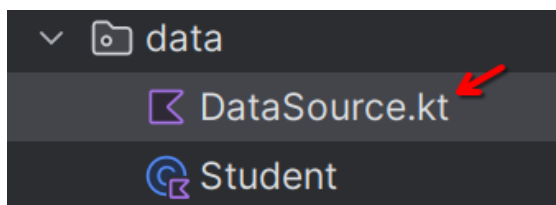
- **@DrawableRes** — аннотация, указывающая что это ID ресурса изображения
- **@StringRes** — аннотация для ID строкового ресурса
- **imageResourceId** — ID изображения студента
- **name** — ID имени студента из **strings.xml**
- **age** — возраст студента (обычное число)
- **description** — ID описания студента

Не забудьте добавить импорты:

```
import androidx.annotation.DrawableRes
import androidx.annotation.StringRes
```

3.3. Создание источника данных

В том же пакете **data** создайте файл **DataSource.kt**:



Класс **DataSource** будет содержать список всех студентов приложения.

Добавим список наших студентов внутрь файла. Пример из двух студентов:

```
val students = listOf(
    Student(
        R.drawable.student1,
        R.string.student1, 19,
        R.string.student1_desc
    ),
    Student(
        R.drawable.student2,
        R.string.student2, 20,
        R.string.student2_desc
    ),
)
```

- **val students** — неизменяемая переменная, содержащая список
- **listOf()** — создаёт список
- Мы указываем в конструкторе: **ссылку на картинку, имя, возраст, описание**

Далее вам нужно добавить по аналогии остальных 8 студентов самостоятельно.

Не забудьте добавить импорт:

```
import com.leontev.student_material_design.R
```

Шаг 4. Создание базового интерфейса

Откройте **MainActivity.kt**.

4.1. Создание функции для отображения иконки студента

Создадим **Composable-функцию StudentIcon()**, которая будет отображать аватарку студента.

1. Создаём функцию, которая принимает ID изображения (studentIcon)

```
@Composable
fun StudentIcon(
    @DrawableRes studentIcon: Int,
    modifier: Modifier = Modifier
) { }
```

2. Внутри используем компонент **Image** для отображения картинки:

```
@Composable
fun StudentIcon(
    ...
) {
    Image(
    )
}
```

3. В компоненте **Image** добавляем:

```
Image(
    modifier = modifier
        .size(dimensionResource(R.dimen.image_size))
        .padding(dimensionResource(R.dimen.padding_small)),
    painter = painterResource(studentIcon),
    contentDescription = null
)
```

- **size()** — устанавливает размер 64dp (из `dimens.xml`)
- **padding()** — добавляет отступ 8dp
- **painterResource()** — загружает изображение по ID

- **contentDescription** = null — изображение декоративное, не требует описания для accessibility

При наличии ошибок, добавьте отсутствующие импорты:

```
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.layout.padding
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.dimensionResource
import androidx.annotation.DrawableRes
```

4.2. Создание функции для отображения информации о студенте

Создадим **Composable-функцию StudentInformation()**, которая будет показывать имя и возраст студента.

1. Создаём функцию, которая принимает ID имени (studentName) и возраст (studentAge):

```
@Composable
fun StudentInformation(
    @StringRes studentName: Int,
    studentAge: Int,
    modifier: Modifier = Modifier
) { }
```

2. Используем Column для вертикального размещения элементов:

```
@Composable
fun StudentInformation(
    ...
) {
    Column(modifier = modifier) {
    }
}
```

3. Первый Text отображает имя студента со стилем displayMedium (крупный текст):

```
Column(modifier = modifier) {
    Text(
        text = stringResource(studentName),
        modifier = Modifier.padding(top = dimensionResource(
            R.dimen.padding_small)),
        style = MaterialTheme.typography.displayMedium
    )
}
```

- *stringResource()* — получаем строку из *strings.xml* по ID
- *MaterialTheme.typography* — применяет стили типографики из темы

4. Второй Text отображает возраст со стилем *bodyLarge* (обычный текст)

```
Column(modifier = modifier) {
    1 Text(...)
    2 Text(
        text = stringResource(R.string.years_old,
            studentAge),
        style = MaterialTheme.typography.bodyLarge
    )
}
```

При наличии ошибок, добавьте отсутствующие импорты:

```
import androidx.compose.foundation.layout.Column
import androidx.compose.material3.Text
import androidx.compose.material3.MaterialTheme
import androidx.compose.ui.res.stringResource
import androidx.annotation.StringRes
```

4.3. Создание карточки студента

Создадим **Composable**-функцию **StudentItem()**, которая объединяет иконку и информацию в одну строку.

1. Создаём функцию, которая принимает объект **Student**

```
@Composable
fun StudentItem(
    student: Student,
    modifier: Modifier = Modifier
) { }
```

2. Используем **Row** для горизонтального размещения элементов

```
@Composable
fun StudentItem(
    ...
) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(dimensionResource(R.dimen.padding_small))
    ) { }
```

- ***fillMaxWidth()*** — элемент занимает всю ширину экрана
- ***padding()*** — добавляет отступ вокруг строки

3. Внутри Row:

```
Row(...) {
    StudentIcon(student.imageResourceId)
    StudentInformation(student.name, student.age)
}
```

- Вызываем ***StudentIcon()*** для отображения аватарки
- Вызываем ***StudentInformation()*** для отображения имени и возраста

При наличии ошибок, добавьте отсутствующие импорты:

```
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.fillMaxWidth
import com.example.studentmaterialdesign.data.Student
```

4.4. Создание списка студентов

Создадим **Composable-функцию StudentApp()**, которая отображает прокручиваемый список всех студентов.

```
@Composable
fun StudentApp() {
    LazyColumn {
        items(students) {
            StudentItem(student = it)
        }
    }
}
```

- Используем **LazyColumn** для создания **вертикального** прокручиваемого списка
- **items(students)** — перебирает каждый элемент из списка **students**
- Для каждого студента (**it**) создаём **StudentItem**
- **LazyColumn** эффективно работает с большими списками — отрисовывает только видимые элементы

При наличии ошибок, добавьте отсутствующие импорты:

```
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import com.example.studentmaterialdesign.data.students
```

4.5. Создание Preview

Создадим функцию для **предварительного просмотра** приложения:

```
@Preview
@Composable
fun StudentPreview() {
    Student_Material_DesignTheme(darkTheme = false) {
        StudentApp()
    }
}
```

- **@Preview** — позволяет увидеть UI без запуска приложения
- **Student_Material_DesignTheme()** — применяет тему приложения
- **darkTheme = false** — используем светлую тему
- Вызываем **StudentApp()** для отображения списка

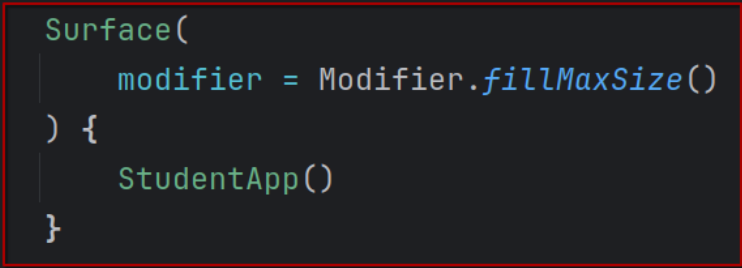
При наличии ошибок, добавьте отсутствующие импорты:

```
import androidx.compose.ui.tooling.preview.Preview
```

4.6. Подключение в MainActivity

Теперь **вызовем** нашу функцию **StudentApp()** в главной **Activity**:

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Student_Material_DesignTheme {
                Surface(
                    modifier = Modifier.fillMaxSize()
                ) {
                    StudentApp()
                }
            }
        }
    }
}
```



- **setContent {}** — устанавливает содержимое Activity
- **Student_Material_DesignTheme {}** — применяет тему Material Design
- **Surface** — создаёт базовую поверхность с фоновым цветом темы
- **StudentApp()** — отображает наш список студентов

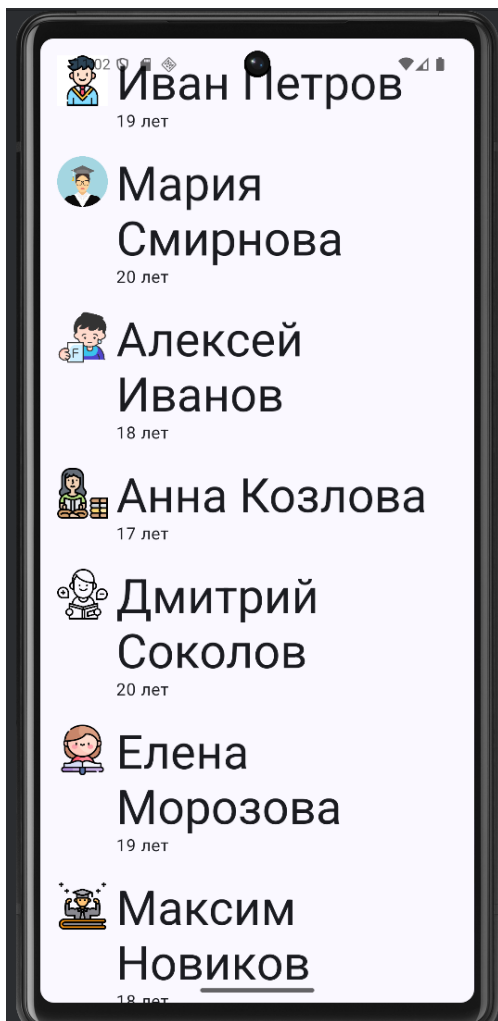
При наличии ошибок, добавьте отсутствующие импорты:

```
import androidx.compose.foundation.layout.fillMaxSize
```

```
import androidx.compose.material3.Surface
```

Шаг 5. Запуск приложения

1. Нажмите **Run**
2. Выберите эмулятор или устройство
3. Приложение должно отобразить список из 10 студентов



На данном этапе приложение использует стандартную тему Material Design.

GitHub

1. Сделайте скриншот запущенного приложения на эмуляторе.
2. Сохраните скриншот в папку `/app/img/`
3. Назовите файл: `step5_app_running_lightTheme_FIO.png`

Commit изменений:

```
git add .
```

```
git commit -m "Step 5: Basic UI completed"
```

```
git push
```

Шаг 6. Добавление цветовой схемы

Теперь мы изменим цветовую схему приложения. **Цветовая схема** — это комбинация цветов, которую использует ваше приложение. Различные цветовые сочетания вызывают разное настроение и влияют на восприятие пользователей.

Теория: Как представлены цвета в Android

В Android цвет представлен **шестнадцатеричным (hex) значением**. Нех-код начинается с символа решётки #, за которым следуют шесть букв и/или цифр, представляющих компоненты **красного, зелёного и синего (RGB)** цветов:

- Первые два символа — **красный (Red)**
- Следующие два — **зелёный (Green)**
- Последние два — **синий (Blue)**

Пример: #FF5722

- FF — красный (максимум)
- 57 — зелёный (средний)
- 22 — синий (минимум)

Цвет также может включать **альфа-значение** (прозрачность):

- #00 — 0% непрозрачности (полностью прозрачный)
- #FF — 100% непрозрачности (полностью непрозрачный)

Когда альфа включена, она указывается **первыми двумя символами** после #.

Примеры цветов:

Цвет	Нех-код	Описание
Красный	#FF0000	Чистый красный
Зелёный	#00FF00	Чистый зелёный
Синий	#0000FF	Чистый синий
Белый	#FFFFFF	Максимум всех цветов
Чёрный	#000000	Отсутствие цвета
Серый	#808080	Средний серый
Прозрачный красный	#80FF0000	Красный с 50% прозрачностью

Роли цветов в Material Design

Material Design определяет следующие **роли цветов**:

- **Primary** — основной цвет для ключевых компонентов (кнопки, заголовки)
- **Secondary** — второстепенный цвет для менее заметных элементов
- **Tertiary** — третичный цвет для контрастных акцентов

- **Он-цвета** — цвет текста/иконок, который отображается **поверх** другого цвета (например, onPrimary — текст на primary фоне)

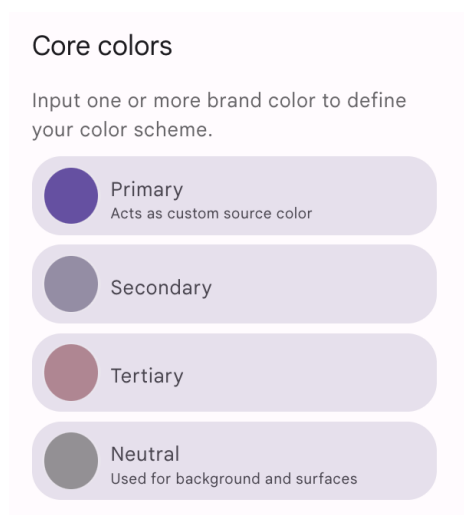
Эти роли обеспечивают **согласованность дизайна**, где связанные компоненты окрашены одинаково.

6.1. Использование Material Theme Builder (опционально)

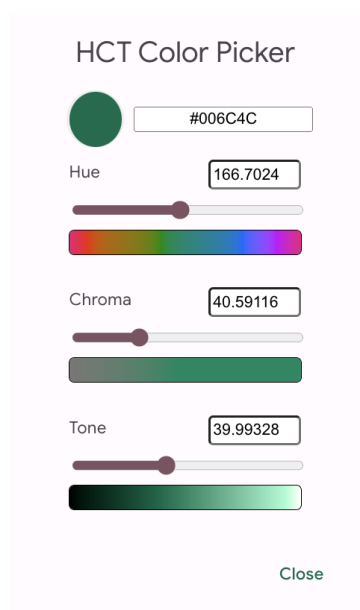
Примечание: В этой лабораторной мы используем готовую цветовую схему, которую **я вам напишу дальше**, но для общего развития и применения своих тем рекомендуется изучить этот шаг сейчас.

Вы можете создать свою цветовую схему с помощью [Material Theme Builder](#):

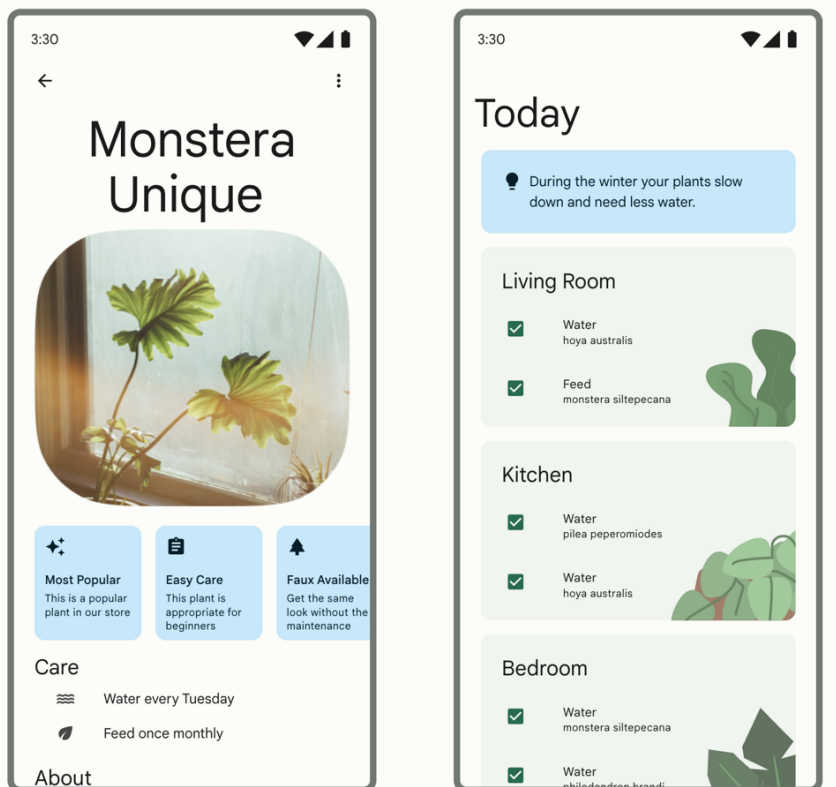
1. Откройте [Material Theme Builder](#)
2. В левой панели найдите **Core Colors** → **Primary**



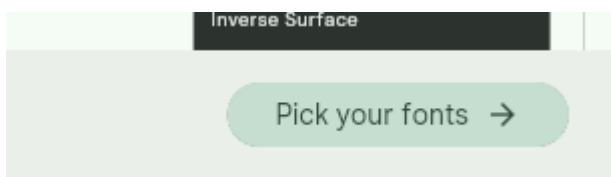
3. В HCT Color Picker введите цвет, например #006C4C (зелёный)



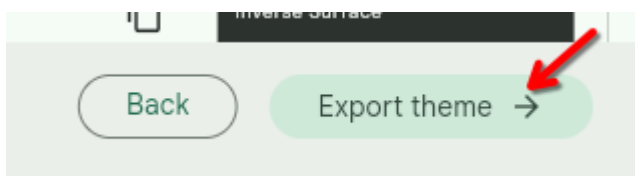
4. Система автоматически сгенерирует полную палитру для светлой и тёмной тем



5. Нажмите **Pick your fonts**

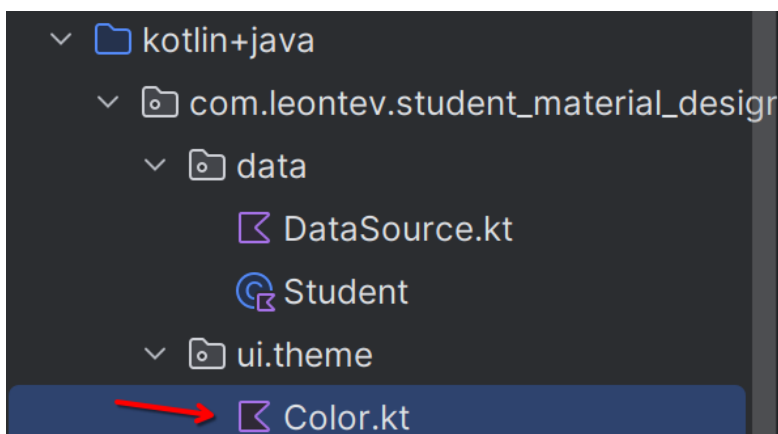


6. Нажмите **Export theme** для скачивания файлов



6.2. Добавление цветовой палитры в Color.kt

Откройте файл **ui/theme/Color.kt** (находится в пакете ui.theme)



Замените содержимое файла на следующий код (**НЕ** изменяя первую строчку с названием вашего пакета!):

тут будет ваш пакет

```
import androidx.compose.ui.graphics.Color

// Светлая тема
val md_theme_light_primary = Color(0xFF006C4C)
val md_theme_light_onPrimary = Color(0xFFFFFFFF)
val md_theme_light_primaryContainer = Color(0xFF89F8C7)
val md_theme_light_onPrimaryContainer = Color(0xFF002114)
val md_theme_light_secondary = Color(0xFF4D6357)
val md_theme_light_onSecondary = Color(0xFFFFFFFF)
val md_theme_light_secondaryContainer = Color(0xFFCFE9D9)
val md_theme_light_onSecondaryContainer = Color(0xFF092016)
val md_theme_light_tertiary = Color(0xFF3D6373)
val md_theme_light_onTertiary = Color(0xFFFFFFFF)
val md_theme_light_tertiaryContainer = Color(0xFFC1E8FB)
val md_theme_light_onTertiaryContainer = Color(0xFF001F29)
val md_theme_light_error = Color(0xFFBA1A1A)
val md_theme_light_errorContainer = Color(0xFFFFDAD6)
val md_theme_light_onError = Color(0xFFFFFFFF)
val md_theme_light_onErrorContainer = Color(0xFF410002)
val md_theme_light_background = Color(0xFFFFBFD9)
val md_theme_light_onBackground = Color(0xFF191C1A)
val md_theme_light_surface = Color(0xFFFFBFD9)
val md_theme_light_onSurface = Color(0xFF191C1A)
val md_theme_light_surfaceVariant = Color(0xFFDBE5DD)
val md_theme_light_onSurfaceVariant = Color(0xFF404943)
val md_theme_light_outline = Color(0xFF707973)
val md_theme_light_inverseOnSurface = Color(0xFFE8F5E9)
val md_theme_light_inverseSurface = Color(0xFF2E312F)
val md_theme_light_inversePrimary = Color(0xFF6CDBC4)
val md_theme_light_shadow = Color(0xFF000000)
val md_theme_light_surfaceTint = Color(0xFF006C4C)
val md_theme_light_outlineVariant = Color(0xFFBFC9C2)
val md_theme_light_scrim = Color(0xFF000000)

// Тёмная тема
val md_theme_dark_primary = Color(0xFF6CDBC4)
val md_theme_dark_onPrimary = Color(0xFF003826)
val md_theme_dark_primaryContainer = Color(0xFF005138)
val md_theme_dark_onPrimaryContainer = Color(0xFF89F8C7)
val md_theme_dark_secondary = Color(0xFFB3CCBE)
val md_theme_dark_onSecondary = Color(0xFF1F352A)
val md_theme_dark_secondaryContainer = Color(0xFF354B40)
val md_theme_dark_onSecondaryContainer = Color(0xFFCFE9D9)
val md_theme_dark_tertiary = Color(0xFFA5CCDF)
val md_theme_dark_onTertiary = Color(0xFF073543)
val md_theme_dark_tertiaryContainer = Color(0xFF244C5B)
val md_theme_dark_onTertiaryContainer = Color(0xFFC1E8FB)
val md_theme_dark_error = Color(0xFFFFB4AB)
val md_theme_dark_errorContainer = Color(0xFF93000A)
val md_theme_dark_onError = Color(0xFF690005)
val md_theme_dark_onErrorContainer = Color(0xFFFFDAD6)
val md_theme_dark_background = Color(0xFF191C1A)
val md_theme_dark_onBackground = Color(0xFFE1E3DF)
val md_theme_dark_surface = Color(0xFF191C1A)
val md_theme_dark_onSurface = Color(0xFFE1E3DF)
val md_theme_dark_surfaceVariant = Color(0xFF404943)
val md_theme_dark_onSurfaceVariant = Color(0xFFBFC9C2)
val md_theme_dark_outline = Color(0xFF8A938C)
val md_theme_dark_inverseOnSurface = Color(0xFF191C1A)
val md_theme_dark_inverseSurface = Color(0xFFE1E3DF)
val md_theme_dark_inversePrimary = Color(0xFF006C4C)
val md_theme_dark_shadow = Color(0xFF000000)
val md_theme_dark_surfaceTint = Color(0xFF6CDBC4)
val md_theme_dark_outlineVariant = Color(0xFF404943)
val md_theme_dark_scrim = Color(0xFF000000)
```

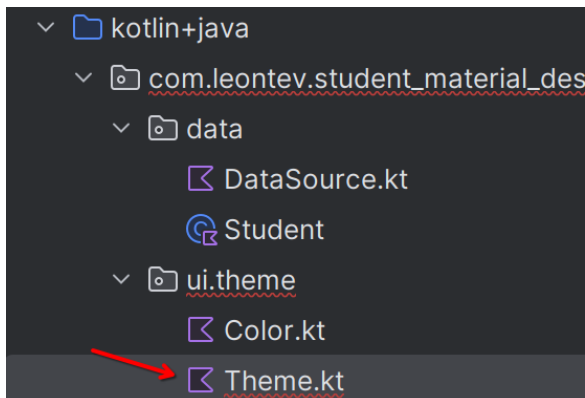
Объяснение:

- Мы определяем цвета для **светлой** (light) и **тёмной** (dark) тем

- Каждый цвет имеет префикс md_theme_ (Material Design theme)
- Основной цвет приложения — зелёный (#006C4C)

6.3. Обновление Theme.kt

Откройте файл **ui/theme/Theme.kt**.



Замените содержимое файла на следующий код (НЕ изменяя первую строчку с названием вашего пакета! и **в функции название вашего приложения**):

```

тут будет ваш пакет
import android.app.Activity
import android.os.Build
import android.view.View
import androidx.compose.foundation.isSystemInDarkTheme
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.darkColorScheme
import androidx.compose.material3.dynamicDarkColorScheme
import androidx.compose.material3.dynamicLightColorScheme
import androidx.compose.material3.lightColorScheme
import androidx.compose.runtime.Composable
import androidx.compose.runtime.SideEffect
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.graphics.toArgb
import androidx.compose.ui.platform.LocalContext
import androidx.compose.ui.platform.LocalView
import androidx.core.view.WindowCompat
private val LightColors = lightColorScheme(
    primary = md_theme_light_primary,
    onPrimary = md_theme_light_onPrimary,
    primaryContainer = md_theme_light_primaryContainer,
    onPrimaryContainer = md_theme_light_onPrimaryContainer,
    secondary = md_theme_light_secondary,
    onSecondary = md_theme_light_onSecondary,
    secondaryContainer = md_theme_light_secondaryContainer,
    onSecondaryContainer = md_theme_light_onSecondaryContainer,
    tertiary = md_theme_light_tertiary,
    onTertiary = md_theme_light_onTertiary,
    tertiaryContainer = md_theme_light_tertiaryContainer,
    onTertiaryContainer = md_theme_light_onTertiaryContainer,
    error = md_theme_light_error,
    errorContainer = md_theme_light_errorContainer,
    onError = md_theme_light_onError,
    onErrorContainer = md_theme_light_onErrorContainer,
    background = md_theme_light_background,
    onBackground = md_theme_light_onBackground,
    surface = md_theme_light_surface,
    onSurface = md_theme_light_onSurface,
    surfaceVariant = md_theme_light_surfaceVariant,
    onSurfaceVariant = md_theme_light_onSurfaceVariant,
    outline = md_theme_light_outline,
    inverseOnSurface = md_theme_light_inverseOnSurface,

```

```

inverseSurface = md_theme_light_inverseSurface,
inversePrimary = md_theme_light_inversePrimary,
surfaceTint = md_theme_light_surfaceTint,
outlineVariant = md_theme_light_outlineVariant,
scrim = md_theme_light_scrim,
)
private val DarkColors = darkColorScheme(
    primary = md_theme_dark_primary,
    onPrimary = md_theme_dark_onPrimary,
    primaryContainer = md_theme_dark_primaryContainer,
    onPrimaryContainer = md_theme_dark_onPrimaryContainer,
    secondary = md_theme_dark_secondary,
    onSecondary = md_theme_dark_onSecondary,
    secondaryContainer = md_theme_dark_secondaryContainer,
    onSecondaryContainer = md_theme_dark_onSecondaryContainer,
    tertiary = md_theme_dark_tertiary,
    onTertiary = md_theme_dark_onTertiary,
    tertiaryContainer = md_theme_dark_tertiaryContainer,
    onTertiaryContainer = md_theme_dark_onTertiaryContainer,
    error = md_theme_dark_error,
    errorContainer = md_theme_dark_errorContainer,
    onError = md_theme_dark_onError,
    onErrorContainer = md_theme_dark_onErrorContainer,
    background = md_theme_dark_background,
    onBackground = md_theme_dark_onBackground,
    surface = md_theme_dark_surface,
    onSurface = md_theme_dark_onSurface,
    surfaceVariant = md_theme_dark_surfaceVariant,
    onSurfaceVariant = md_theme_dark_onSurfaceVariant,
    outline = md_theme_dark_outline,
    inverseOnSurface = md_theme_dark_inverseOnSurface,
    inverseSurface = md_theme_dark_inverseSurface,
    inversePrimary = md_theme_dark_inversePrimary,
    surfaceTint = md_theme_dark_surfaceTint,
    outlineVariant = md_theme_dark_outlineVariant,
    scrim = md_theme_dark_scrim,
)
@Composable
fun Student_Material_DesignTheme( // ТУТ ВНИМАТЕЛЬНО ДОЛЖНО БЫТЬ НАЗВАНИЕ ВАШЕГО ПРИЛОЖЕНИЯ
    darkTheme: Boolean = isSystemInDarkTheme(),
    // Dynamic color is available on Android 12+
    dynamicColor: Boolean = false,
    content: @Composable () -> Unit
) {
    val colorScheme = when {
        dynamicColor && Build.VERSION.SDK_INT >= Build.VERSION_CODES.S -> {
            val context = LocalContext.current
            if (darkTheme) dynamicDarkColorScheme(context) else dynamicLightColorScheme(context)
        }

        darkTheme -> DarkColors
        else -> LightColors
    }
    val view = LocalView.current
    if (!view.isInEditMode) {
        SideEffect {
            setUpEdgeToEdge(view, darkTheme)
        }
    }

    MaterialTheme(
        colorScheme = colorScheme,
        typography = Typography,
        content = content
    )
}
private fun setUpEdgeToEdge(view: View, darkTheme: Boolean) {
    val window = (view.context as Activity).window
    WindowCompat.setDecorFitsSystemWindows(window, false)
    window.statusBarColor = Color.Transparent.toArgb()
}

```

```

val navigationBarColor = when {
    Build.VERSION.SDK_INT >= 29 -> Color.Transparent.toArgb()
    Build.VERSION.SDK_INT >= 26 -> Color(0xFF, 0xFF, 0xFF, 0x63).toArgb()
    else -> Color(0x00, 0x00, 0x00, 0x50).toArgb()
}
window.navigationBarColor = navigationBarColor
val controller = WindowCompat.getInsetsController(window, view)
controller.isAppearanceLightStatusBars = !darkTheme
controller.isAppearanceLightNavigationBars = !darkTheme
}

```

Объяснение работы colorScheme:

В функции `Student_Material_DesignTheme()` переменная `colorScheme` использует оператор `when`:

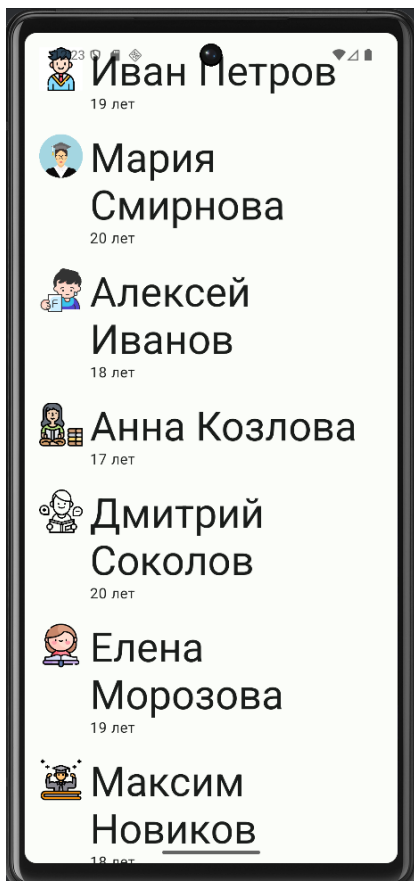
1. Если **dynamicColor = true** и версия Android ≥ 12 (SDK S):

- Проверяет, включена ли тёмная тема
- Устанавливает `dynamicDarkColorScheme` или `dynamicLightColorScheme`

2. Если не используется **Dynamic Color**:

- Проверяет `darkTheme`
- Если `true` → `DarkColors`
- Если `false` → `LightColors`

По умолчанию `dynamicColor = false`, поэтому приложение использует наши кастомные цвета. Перезапустите ваше приложение в эмуляторе, и удостоверьтесь, что фоновый цвет поменялся:



6.4. Добавление Card для элементов списка

Теперь обернём каждый элемент списка в **Card**, чтобы выделить элементы на фоне.

1. Откройте **MainActivity.kt** и найдите функцию `StudentItem()`.

2. Оберните **Row** в **Card**:

```
Card() {  
    Row(...) {  
        StudentIcon(student.imageResourceId)  
        StudentInformation(student.name, student.age)  
    }  
}
```

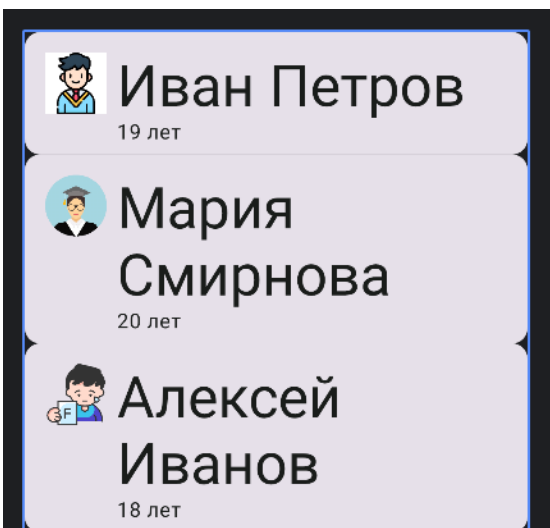
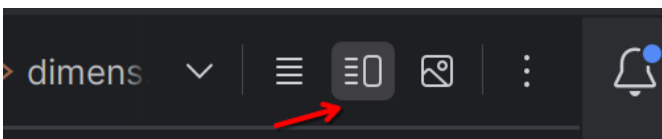
- **Row** получает новый экземпляр **Modifier**
- **Card** автоматически применяет цвет `surface` из темы

3. Примените модификатор для **Card**, чтобы он передавался в него, а не в **Row**:

```
@Composable  
fun StudentItem(  
    student: Student,  
    modifier: Modifier = Modifier  
) {  
    Card(modifier = modifier){Row(  

```

4. В предварительном просмотре, проверьте что изменилось отображение:



6.5. Добавление отступов между элементами

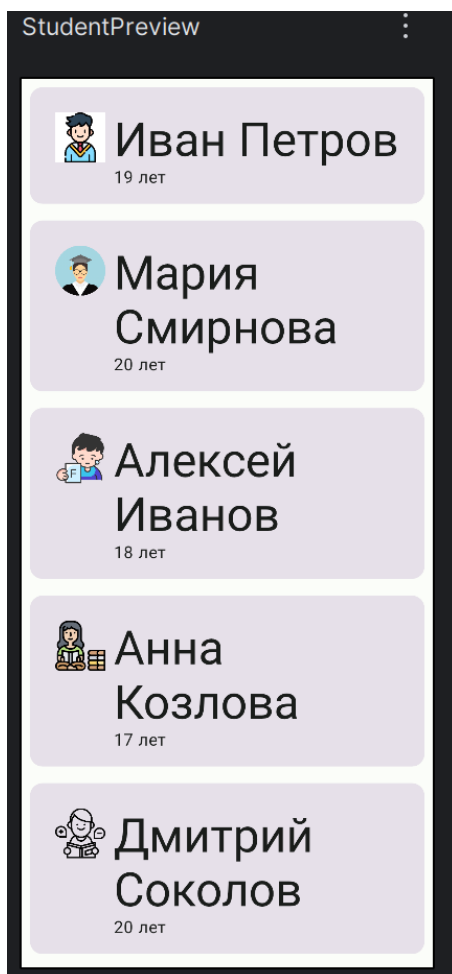
Обновим функцию **StudentApp()**, чтобы добавить отступы между карточками.

Замените функцию **StudentApp()**:

```
@Composable
fun StudentApp() {
    Scaffold { it ->
        LazyColumn(contentPadding = it) {
            items(students) {
                StudentItem(
                    student = it,
                    modifier = Modifier
                        .padding(dimensionResource(R.dimen.padding_small))
                )
            }
        }
    }
}
```

- Добавили **modifier** с отступом **padding_small** (8dp) для каждого элемента
- Теперь между карточками есть визуальное разделение

В предварительном просмотре, проверьте что изменилось отображение:



GitHub

1. Сделайте скриншот запущенного приложения в **Preview**.
2. Сохраните скриншот в папку **/app/img/**
3. Назовите файл: **step6_colors_applied_FIO.png**

Commit изменений:

git add .

git commit -m "Step 6: Custom color scheme added"

git push

Шаг 7. Поддержка тёмной темы

Тёмная тема использует более тёмные, приглушённые цвета и:

- **Снижает энергопотребление** (на OLED-экранах)
- **Улучшает видимость** для пользователей с плохим зрением
- **улучшает просмотр на устройстве** в условиях низкой освещённости

Android поддерживает **Force Dark** (автоматическую тёмную тему), но лучше реализовать её самостоятельно для полного контроля.

7.1. Просмотр тёмной темы в Preview

Мы уже добавили цвета для тёмной темы в предыдущем шаге. Теперь создадим **дополнительный Preview**, чтобы одновременно видеть светлую и тёмную темы.

Откройте **MainActivity.kt**.

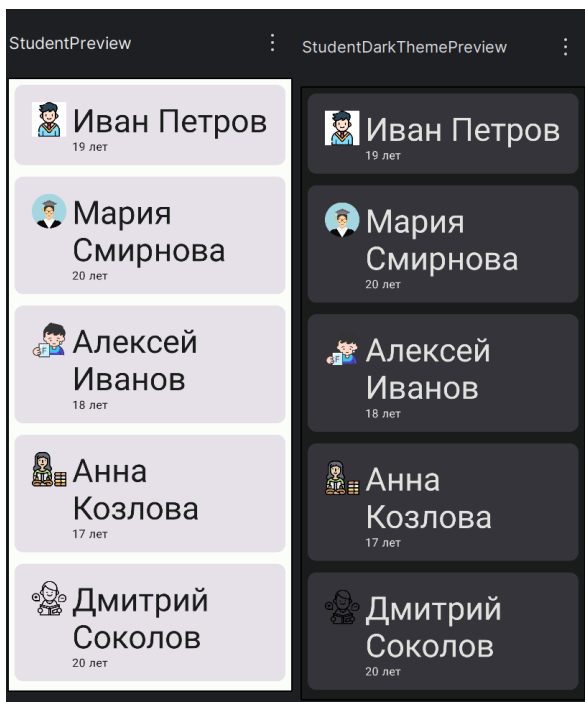
Создайте новую **Preview-функцию** под **StudentPreview()**:

```
@Preview
@Composable
fun StudentDarkThemePreview() {
    Student_Material_DesignTheme(darkTheme = true) {
        StudentApp()
    }
}
```

@Preview — создаём превью

- **darkTheme = true** — включаем тёмную тему
- **StudentApp()** — отображаем список студентов

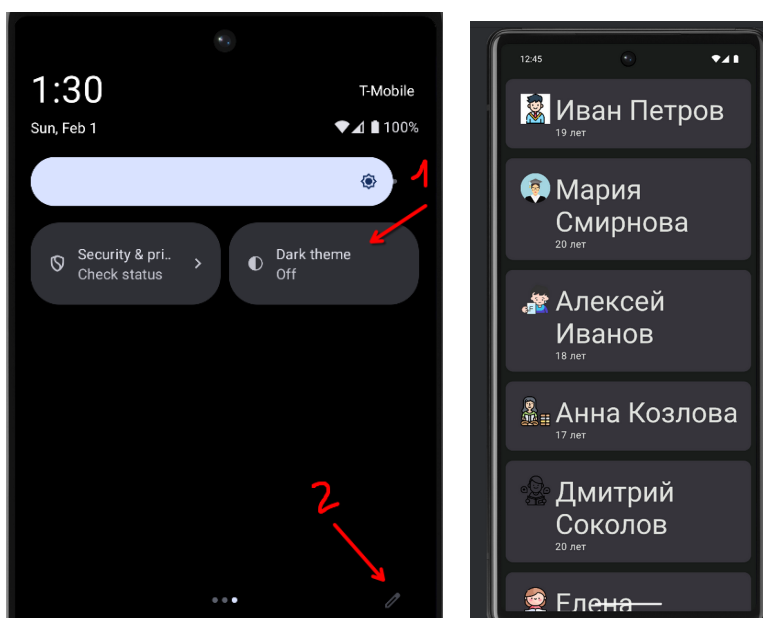
Теперь в панели **Design** прокрутите вниз — вы увидите приложение в тёмной теме с тёмным фоном и светлым текстом:



7.2. Просмотр тёмной темы на устройстве

Чтобы увидеть приложение в тёмной теме на эмуляторе или реальном устройстве:

1. Откройте **Настройки** (Settings) на устройстве
2. Найдите "**Тёмная тема**" (Dark theme) или "**Оформление**" (Display)
3. Включите переключатель "**Тёмная тема**"
4. Переоткройте приложение **Student List** — оно будет в тёмной теме!



GitHub

1. Сделайте скриншот запущенного приложения в **Preview** и два скриншота в эмуляторе (светлая и тёмная тема). **Тёмная тема не работает в старых версиях Android!!! Поэтому если у вас она отсутствует, можете сделать только светлую.**
2. Сохраните скриншот в папку `/app/img/`

3. Назовите файлы:

step7_preview_FIO.png

step7_emulator_lightTheme_FIO.png

step7_emulator_darkTheme_FIO.png

Commit изменений:

git add .

git commit -m "Step 7: Dark theme support added"

git push

Для продолжения работы:

1. Вернитесь в **Настройки**
2. **Отключите** тёмную тему
3. **Продолжайте работу в светлой теме**

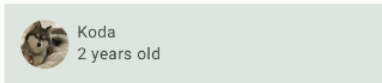
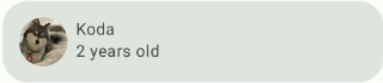
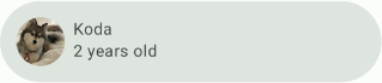
Шаг 8. Добавление форм (Shape)

Применение форм может кардинально изменить внешний вид компонента. **Формы** направляют внимание, идентифицируют компоненты, передают состояние и выражают стиль бренда.

Теория: RoundedCornerShape

Большинство форм определяются с помощью **RoundedCornerShape** — это прямоугольник со скруглёнными углами. Число, переданное в параметр, определяет **степень скругления**:

- **RoundedCornerShape(0.dp)** — прямоугольник без скруглений
- **RoundedCornerShape(50.dp)** — полностью круглые углы (круг)
- **RoundedCornerShape(16.dp)** — умеренное скругление

0.dp	25.dp	50.dp
		

Вы также можете настроить **разное скругление для каждого угла**:

Top left: 50.dp Bottom left: 25.dp Top right: 0.dp Bottom right: 15.dp	Top left: 15.dp Bottom left: 50.dp Top right: 50.dp Bottom right: 15.dp	Top left: 0.dp Bottom left: 50.dp Top right: 0.dp Bottom right: 50.dp

Категории размеров компонентов

Material Design определяет **три категории размеров** компонентов:

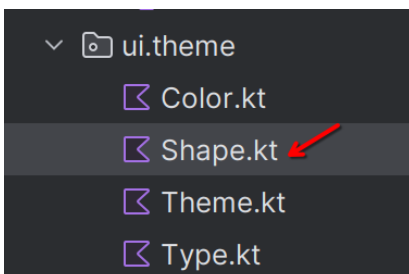
- **Small** — маленькие элементы (чипы, маленькие кнопки)
- **Medium** — средние элементы (карточки, кнопки)
- **Large** — большие элементы (диалоги, большие карточки)

Файл **Shape.kt** используется для определения форм этих компонентов.

8.1. Создание файла Shape.kt

Создадим файл для определения форм компонентов.

1. Кликните правой кнопкой на папке **ui/theme**
2. **New** → **Kotlin Class/File**
3. Выберите **File**
4. Имя: **Shape**



Добавьте следующий код (**Примечание:** Первая строка пакета будет отличаться — используйте название **вашего** пакета!):

```
package com.leontev.student_material_design.ui.theme

import androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material3.Shapes
import androidx.compose.ui.unit.dp

val Shapes = Shapes(
    small = RoundedCornerShape(50.dp),
)
```

- Создаём объект **Shapes** — набор форм для приложения

- Устанавливаем `small = RoundedCornerShape(50.dp)` — это создаст **круглую форму**
- `50.dp` скругление делает углы полностью круглыми (для квадратного изображения получится круг)

За что отвечает этот файл:

- Определяет формы всех компонентов приложения
- `small` будет использоваться для маленьких элементов (например, иконок студентов)
- Позже добавим `medium` для карточек

8.2. Применение круглой формы к изображению студента

Теперь сделаем аватарки студентов **круглыми**.

Откройте **MainActivity.kt** и найдите функцию `StudentIcon()`.

Добавьте модификатор `.clip()` к `Image`:

```
Image(  
    modifier = modifier  
        .size(dimensionResource(R.dimen.image_size))  
        .padding(dimensionResource(R.dimen.padding_small))  
        +.clip(MaterialTheme.shapes.small),
```

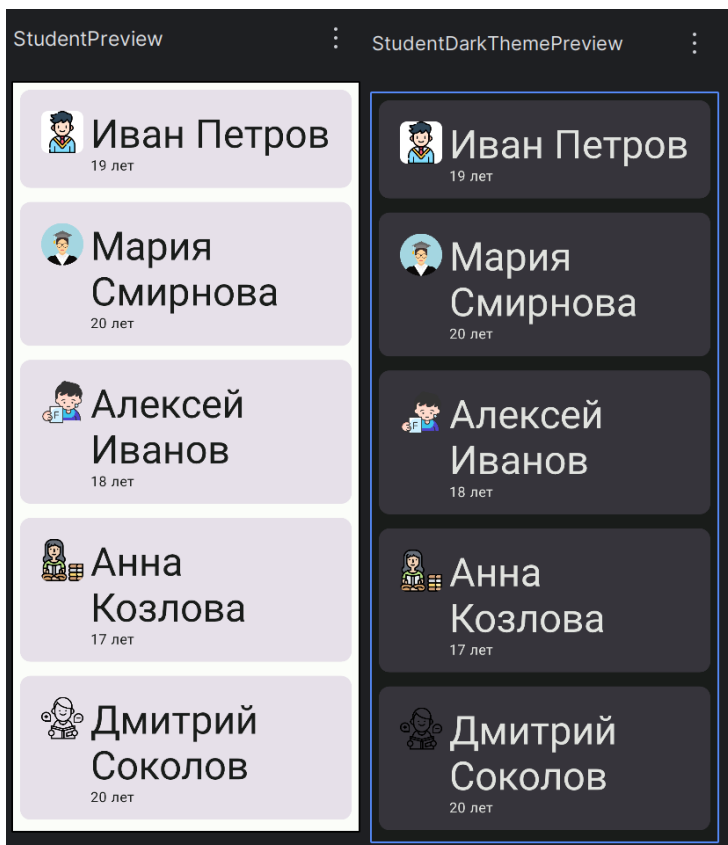
Добавили `.clip(MaterialTheme.shapes.small)` — это **обрезает** изображение по форме `small`

- **MaterialTheme.shapes.small** берёт форму из нашего файла **Shape.kt**
- Изображения теперь круглые!

Необходимый импорт:

```
import androidx.compose.ui.draw.clip
```

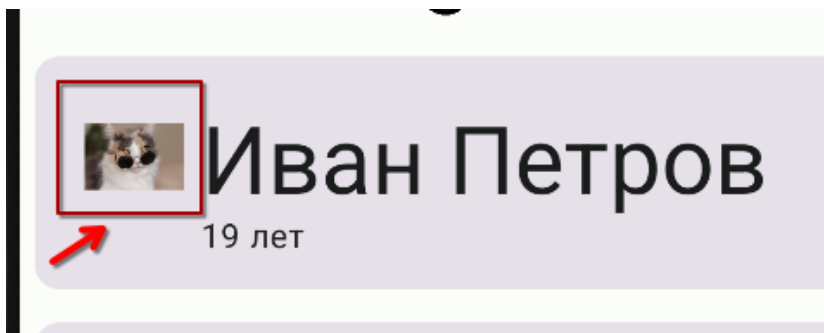
При использовании иконок, сильных изменений вы не увидите. А вот для фотографий или изображений у вас возможно появится проблема, которую мы решим в следующем шаге.



8.3. Обрезка изображений (ContentScale.Crop)

Проблема: Если вы используете прямоугольные фотографии вместо иконок, изображения могут обрезаться по бокам и не выглядеть как полные круги.

Пример проблемы:



Решение: Используйте **ContentScale.Crop** для правильного кадрирования.

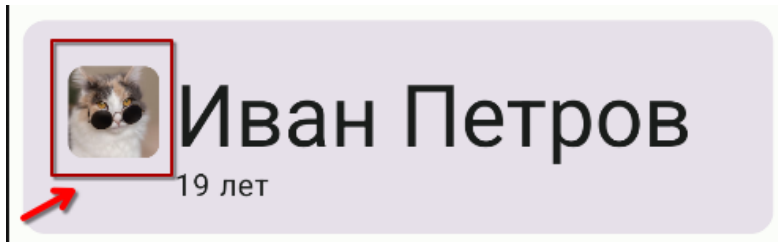
В функции **StudentIcon()** добавьте параметр **contentScale**:

```
Image(  
    modifier = modifier  
        .size(dimensionResource(R.dimen.image_size))  
        .padding(dimensionResource(R.dimen.padding_small))  
        .clip(MaterialTheme.shapes.small),  
    + contentScale = ContentScale.Crop,  
    painter = painterResource(studentIcon),  
    contentDescription = null
```


Необходимый импорт:

```
import androidx.compose.ui.layout.ContentScale
```

Итог:



Что делает `ContentScale.Crop`:

- Масштабирует изображение так, чтобы оно заполнило всю область
- Обрезает лишние части, если пропорции не совпадают
- Гарантирует, что изображение всегда заполняет круг полностью

8.4. Добавление формы для карточек списка

Теперь добавим **скруглённые углы** для карточек студентов.

1: Подключение форм в теме

1. Откройте файл `ui/theme/Theme.kt`
2. Найдите функцию `Student_Material_DesignTheme()`
3. Внутри `MaterialTheme()` добавьте параметр `shapes`:

```
108         MaterialTheme(  
109             colorScheme = colorScheme,  
110             typography = Typography,  
111             + shapes = Shapes,  
112             content = content  
113         )
```

- Подключили наш объект `Shapes` к теме
- Теперь все компоненты будут использовать наши формы

2: Определение формы для карточек

1. Откройте `ui/theme/Shape.kt`
2. Добавьте параметр `medium` в объект `Shapes`:

```
val Shapes = Shapes(  
    small = RoundedCornerShape(50.dp),  
    + medium = RoundedCornerShape(bottomStart = 16.dp, topEnd = 16.dp)  
)
```

Card — это компонент **среднего размера (medium)**

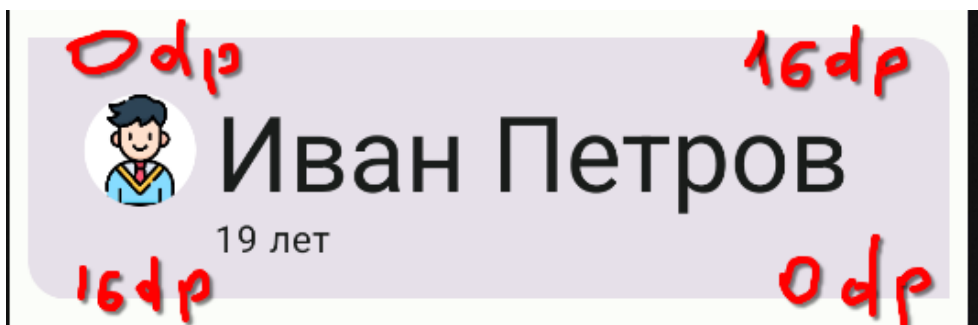
- Устанавливаем скругление **только для двух углов**:
 - **bottomStart = 16.dp** — левый нижний угол
 - **topEnd = 16.dp** — правый верхний угол
- Два других угла остаются острыми (0dp)

Почему именно эти углы?

- Создаётся **диагональный эффект** скругления
- Делает карточки более **динамичными и современными**
- Отличает их от обычных прямоугольных карточек

Объяснение параметров:

- **topStart** — левый верхний угол
- **topEnd** — правый верхний угол
- **bottomStart** — левый нижний угол
- **bottomEnd** — правый нижний угол



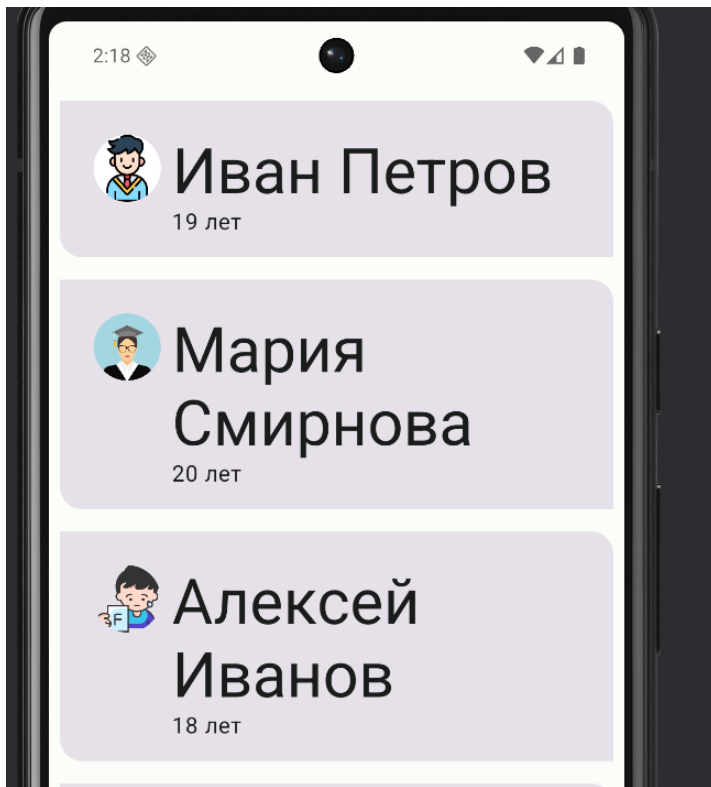
8.5. Автоматическое применение формы к Card

Важно: Компонент **Card** **автоматически** использует форму **medium** из темы, поэтому нам **не нужно явно указывать** форму в коде!

Android Studio применит форму сам.

8.6. Запуск приложения

1. Перезапустите приложение.
2. Проверьте результат:
 - Аватарки студентов — круглые
 - Карточки имеют скруглённые углы (правый верхний и левый нижний)
 - Изображения правильно обрезаны (если используете фотографии)



GitHub

1. Сделайте скриншот с применёнными формами.
2. Сохраните скриншот в папку `/app/img/`
3. Назовите файл: `step8_shapes_applied_lightTheme_FIO.png`

Commit изменений:

git add .

git commit -m "Step 8: Custom shapes added"

git push

Шаг 9. Добавление типографики (Typography)

Теория: Типографическая шкала Material Design

Типографическая шкала — это набор стилей шрифтов, которые можно использовать в приложении, обеспечивая гибкий, но согласованный стиль. **Material Design** включает **пятнадцать стилей шрифтов**, поддерживаемых системой типографики.

Стили сгруппированы в пять категорий с тремя размерами (large, medium, small):

Display (Заголовки-дисплеи)

- Самый крупный текст на экране
- Используется для коротких, важных надписей или чисел

- Лучше всего работает на больших экранах

Headline (Заголовки)

- Подходят для короткого текста с высоким акцентом на маленьких экранах
- Используются для обозначения основных разделов текста или важных областей контента

Title (Названия)

- Меньше заголовков (Headline)
- Используются для текста средней важности, который остаётся относительно коротким

Body (Основной текст)

- Используется для длинных блоков текста в приложении
- Основное содержимое

Label (Метки)

- Самые маленькие служебные стили
- Используются для текста внутри компонентов или очень мелкого текста (подписи)

Таблица типографических стилей

Display Large	Roboto 57/64 . 0
Display Medium	Roboto 45/52 . 0
Display Small	Roboto 36/44 . 0
Headline Large	Roboto 32/40 . 0
Headline Medium	Roboto 28/36 . 0
Headline Small	Roboto 24/32 . 0
Title Large	NEW - Roboto Medium 22/28 . 0
Title Medium	Roboto Medium 16/24 . +0.15
Title Small	Roboto Medium 14/20 . +0.1
Body Large	Roboto 16/24 . +0.5
Body Medium	Roboto 14/20 . +0.25
Body Small	Roboto 12/16 . +0.4
Label Large	Roboto Medium 14/20 . +0.1
Label Medium	Roboto Medium 12/16 . +0.5
Label Small	NEW - Roboto Medium 11/16 . +0.5

Стиль	Размер	Применение
displayLarge	57sp	Главные заголовки
displayMedium	45sp	Крупные заголовки
displaySmall	36sp	Заголовки экранов
headlineLarge	32sp	Заголовки разделов
titleLarge	22sp	Названия карточек
bodyLarge	16sp	Основной текст
labelSmall	11sp	Подписи, метки

9.1. Создание папки для шрифтов

Прежде чем добавить шрифты в приложение, создадим специальную папку.

1. В **Project view** кликните правой кнопкой на папке **res**
2. Выберите **New** → **Android Resource Directory**
3. **Resource type:** выберите font
4. **Directory name:** автоматически установится font
5. Нажмите **OK**

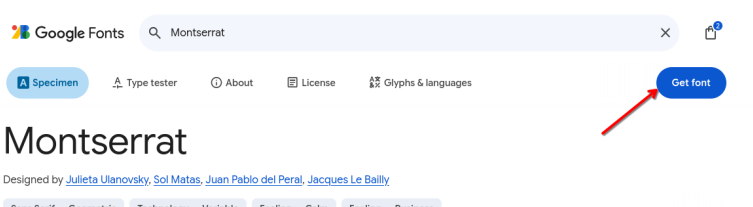
Откройте новую папку шрифтов: **res** → **font**

9.2. Скачивание пользовательских шрифтов

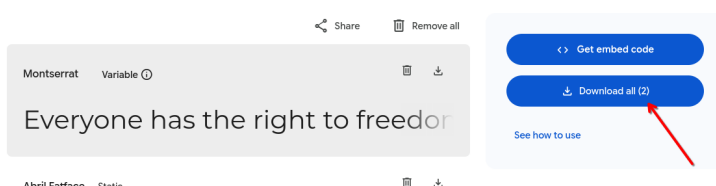
Мы будем использовать шрифты **Montserrat** и **Abril Fatface** из Google Fonts. Если у вас будут проблемы со скачиванием, то к лабораторной работе прикрепляется архив со шрифтами.

Скачивание Montserrat

1. Перейдите на [Google Fonts](https://fonts.google.com/specimen/Montserrat)
2. В поиске введите "**Montserrat**"
3. Нажмите **Get Font** и после **Download family**



2 font families selected



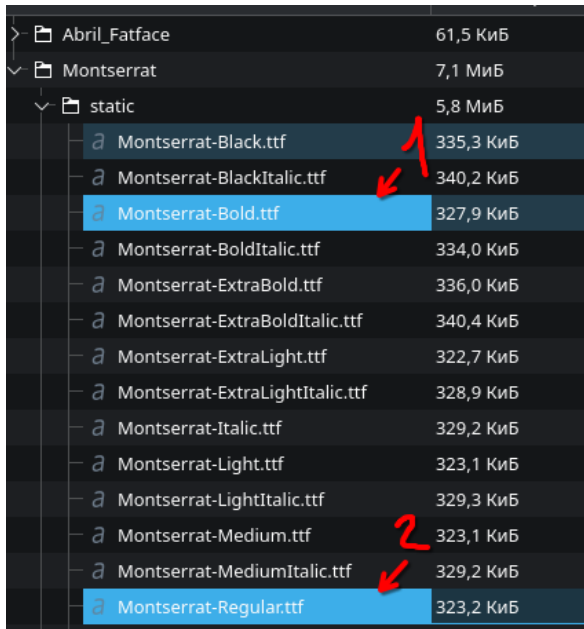
Everyone has the right to freedom of

4. Распакуйте ZIP-архив

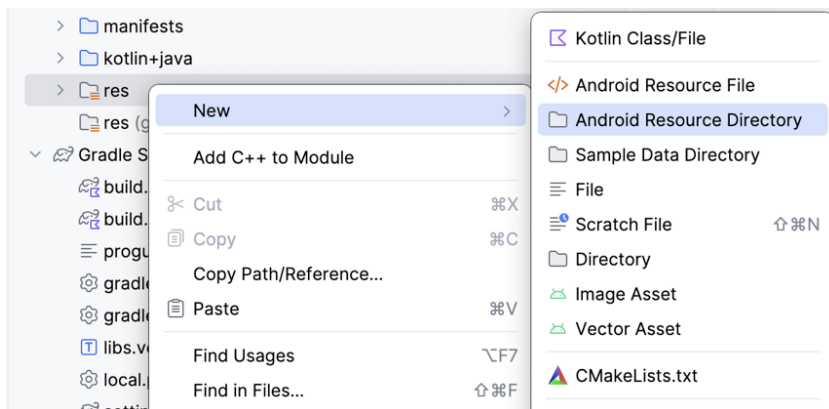
5. Откройте папку **Montserrat** → **static**

6. Найдите файлы:

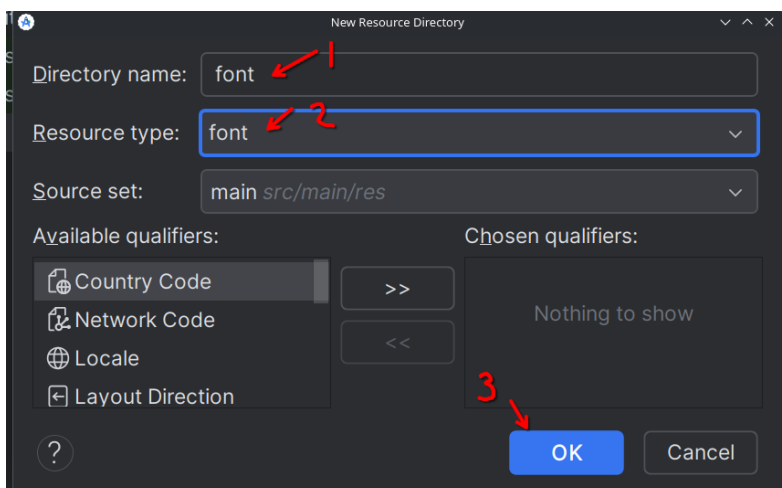
- Montserrat-Bold.ttf
- Montserrat-Regular.ttf



7. Создайте в папке **res** новую папку с именем **font**:



8. Вводите имя **font**, тип **font** и нажмите **OK**:



9. **Перетащите** оба файла в папку **res/font** в Android Studio

10. **Переименуйте** файлы:

- Montserrat-Bold.ttf → **montserrat_bold.ttf**
- Montserrat-Regular.ttf → **montserrat_regular.ttf**

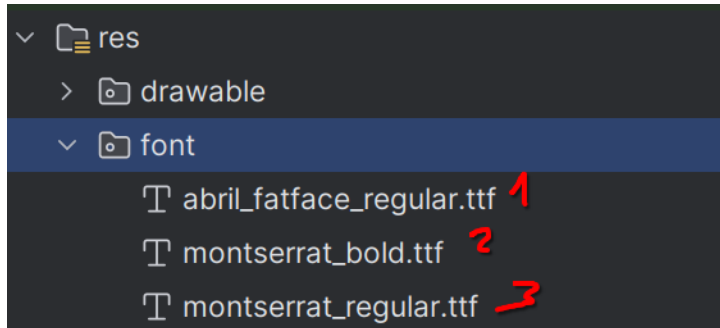
Почему меняем имена?

- Android требует имена файлов **только в нижнем регистре**
- Без пробелов и специальных символов
- Разрешены: a-z, 0-9, _

Скачивание Abril Fatface

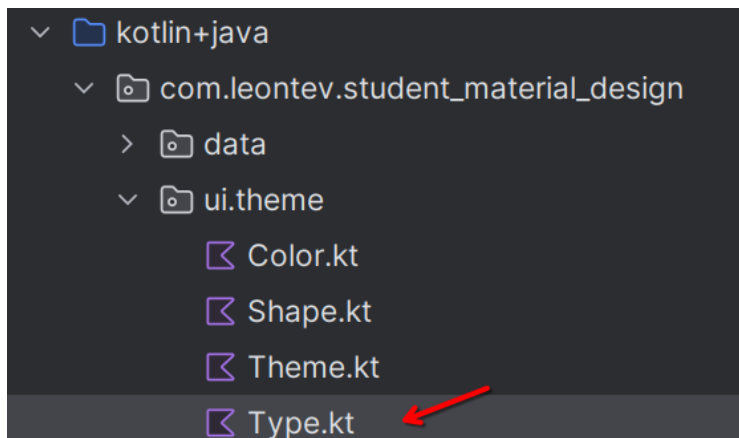
1. На [Google Fonts](#) найдите "**Abril Fatface**"
2. Нажмите **Download family**
3. Откройте скачанную папку **Abril_Fatface**
4. Найдите файл AbrilFatface-Regular.ttf
5. **Перетащите** его в папку **res/font**
6. **Переименуйте** в **abril_fatface_regular.ttf**

Итоговая структура папки font:



9.3. Инициализация шрифтов в Type.kt

Откройте файл **ui/theme/Type.kt**.



Инициализация Abril Fatface

Под импортами, перед объектом Typography, добавьте:

```
val AbrilFatface = FontFamily(  
    Font(R.font.abril_fatface_regular)  
)
```

- Создаём переменную **AbrilFatface** типа **FontFamily**
- **FontFamily()** — группирует различные варианты одного шрифта
- **Font(R.font.abril_fatface_regular)** — загружает файл шрифта

Добавьте необходимые импорты:

```
import androidx.compose.ui.text.font.Font  
import com.leontev.student_material_design.R
```

Обратите внимание импорт R.font у вас будет с вашим названием.

Инициализация Montserrat

Под **Abril Fatface** добавьте:

```
val Montserrat = FontFamily(  
    Font(R.font.montserrat_regular),  
    Font(R.font.montserrat_bold, FontWeight.Bold)  
)
```

- Создаём **FontFamily** для **Montserrat**
- Добавляем два варианта шрифта:
 - Обычный (**montserrat_regular**)
 - Жирный (**montserrat_bold**) с указанием **FontWeight.Bold**

Почему указываем FontWeight.Bold?

- **Compose** не знает автоматически, что файл жирный
- Нужно **явно связать** файл с весом Bold
- Теперь при использовании **fontWeight = FontWeight.Bold** будет применяться правильный файл

9.4. Настройка типографических стилей

Теперь настроим конкретные текстовые стили для приложения.

В этом приложении мы определим **четыре** стиля:

Стиль	Шрифт	Вес	Размер	Применение
displayLarge	Abril Fatface	Normal	36sp	Большие заголовки
displayMedium	Montserrat	Bold	20sp	Имя студента
labelSmall	Montserrat	Bold	14sp	Метки, кнопки
bodyLarge	Montserrat	Regular	14sp	Возраст, описание

Обновление объекта Typography

Замените объект **Typography** в **Type.kt**:

```

21  val Typography = Typography(
22    >    bodyLarge = TextStyle(...)
29    >    / Other default text styles to override .../
45  )

```

На:

```

val Typography = Typography(
    displayLarge = TextStyle(
        fontFamily = AbrilFatface,
        fontWeight = FontWeight.Normal,
        fontSize = 36.sp
    ),
    displayMedium = TextStyle(
        fontFamily = Montserrat,
        fontWeight = FontWeight.Bold,
        fontSize = 20.sp
    ),
    labelSmall = TextStyle(
        fontFamily = Montserrat,
        fontWeight = FontWeight.Bold,
        fontSize = 14.sp
    ),
    bodyLarge = TextStyle(
        fontFamily = Montserrat,
        fontWeight = FontWeight.Normal,
        fontSize = 14.sp
    )
)

```

Объяснение каждого стиля:

displayLarge:

- Шрифт: Abril Fatface (декоративный, выразительный)
- Вес: Normal
- Размер: 36sp (крупный)
- Использование: Главные заголовки приложения

displayMedium:

- Шрифт: Montserrat (современный, читабельный)
- Вес: Bold (жирный)
- Размер: 20sp
- Использование: **Имя студента** в карточке

labelSmall:

- Шрифт: Montserrat
- Вес: Bold
- Размер: 14sp
- Использование: Метки, кнопки (будет использовано позже)

bodyLarge:

- Шрифт: Montserrat
- Вес: Normal (обычный)
- Размер: 14sp
- Использование: **Возраст студента**, основной текст

9.5. Подключение Typography к теме

Откройте `ui/theme/Theme.kt`.

Найдите функцию `Student_Material_DesignTheme()` и убедитесь, что в `MaterialTheme()` указан параметр `typography`:

```
MaterialTheme(  
    colorScheme = colorScheme,  
    typography = Typography, ✓  
    shapes = Shapes,  
    content = content  
)
```

Если строка `typography = Typography` отсутствует — добавьте её!

Это подключает все наши стили типографики к теме приложения.

9.6. Применение типографики к тексту

Теперь применим стили к тексту в приложении.

Откройте **MainActivity.kt** и найдите функцию `StudentInformation()`.

Проверьте что функцию `StudentInformation()` соответствует примеру:

```
@Composable
fun StudentInformation(
    @StringRes studentName: Int,
    studentAge: Int,
    modifier: Modifier = Modifier
) {
    Column(modifier = modifier) {
        Text(
            text = stringResource(studentName),
            modifier = Modifier.padding(top = dimensionResource(R.dimen.padding_small)),
            style = MaterialTheme.typography.displayMedium
        )
        Text(
            text = stringResource(R.string.years_old, studentAge),
            style = MaterialTheme.typography.bodyLarge
        )
    }
}
```

Для имени студента:

- `style = MaterialTheme.typography.displayMedium`
- Имя отображается **жирным Montserrat** размером **20sp**

Для возраста:

- `style = MaterialTheme.typography.bodyLarge`
- Возраст отображается **обычным Montserrat** размером **14sp**

Почему `MaterialTheme.typography`?

- `MaterialTheme` — даёт доступ к теме приложения
- `.typography` — получает типографические стили
- `.displayMedium` / `.bodyLarge` — конкретные стили, которые мы определили

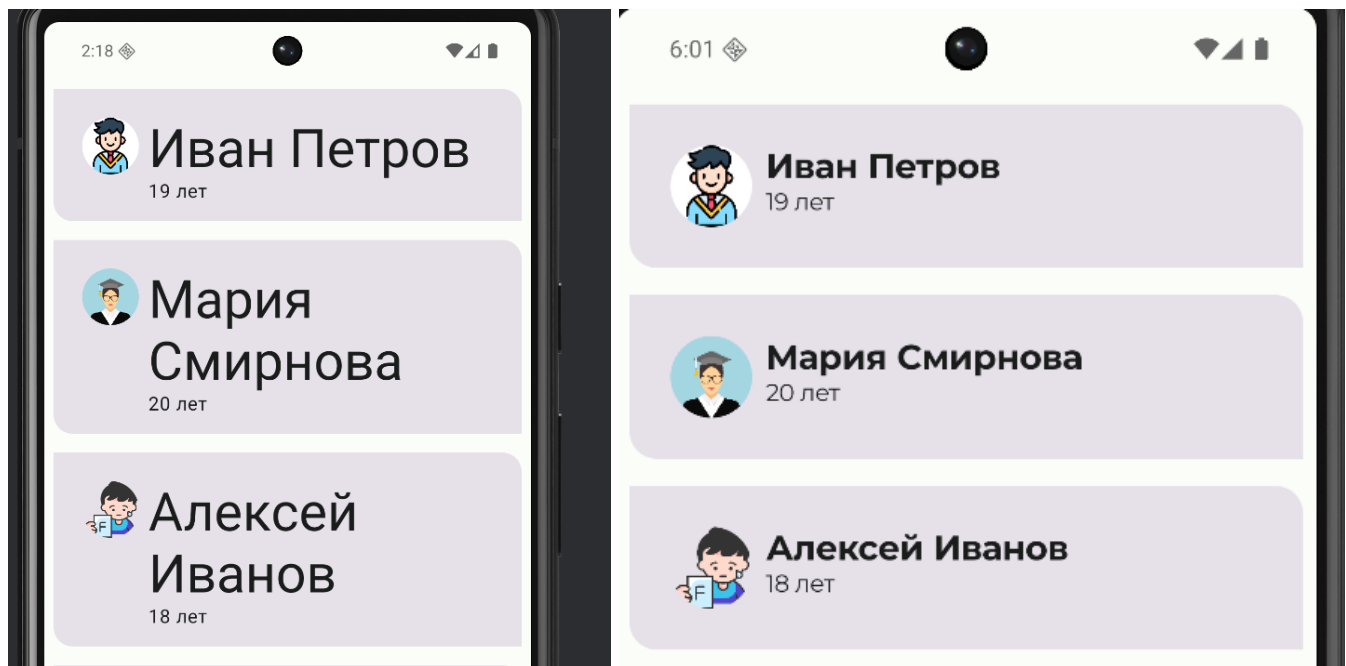
9.7. Проверка результата

Откройте `StudentPreview()` и посмотрите на изменения:

- **Имя студента** — жирный Montserrat, крупнее
- **Возраст** — обычный Montserrat, мельче
- Чёткая **визуальная иерархия** между элементами

9.8. Запуск приложения

1. Нажмите **Run**
2. Проверьте приложение на устройстве
3. Убедитесь, что:
 - Имена студентов отображаются жирным шрифтом
 - Возраст мельче и светлее
 - Шрифты загружены правильно



Если шрифты не отображаются:

- Проверьте названия файлов (должны быть строчными)
- Убедитесь, что файлы в папке `res/font`
- Пересоберите проект: **Build** → **Rebuild Project**

GitHub

1. Сделайте скриншот с кастомными шрифтами
2. Сохраните скриншот в папку `/app/img/`
3. Назовите файл: `step9_typography_applied_lightTheme_FIO.png`

Commit изменений:

git add .

git commit -m "Step 9: Custom typography added"

git push

Шаг 10. Добавление верхней панели приложения (Top App Bar)

Теория: Scaffold и TopAppBar

Scaffold — это layout-компонент, который предоставляет **слоты** для различных элементов экрана, таких как:

- TopAppBar (верхняя панель)
- BottomBar (нижняя панель)
- FloatingActionButton (плавающая кнопка)
- Основной контент

TopAppBar используется для:

- **Брендинга** приложения
- Отображения **названия** приложения
- Навигации
- Действий (кнопки, меню)

Существует **четыре типа** TopAppBar:

- **CenterAligned** — центрированная (используем в этой лабораторной)
- **Small** — маленькая
- **Medium** — средняя
- **Large** — большая

В этом шаге мы создадим верхнюю панель с **логотипом** и **названием** приложения.

10.1. Подготовка иконки для TopAppBar

Прежде чем создавать панель, добавим иконку логотипа.

Вариант 1: Использование готовой иконки (рекомендуется)

Скачайте готовую иконку для приложения:

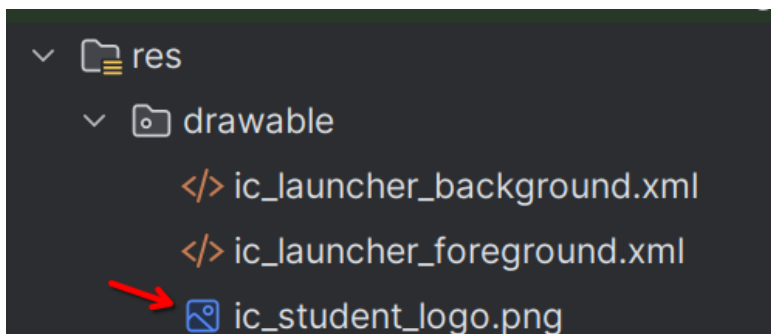
- **Название:** ic_student_logo.png
- **Размер:** 64x64dp
- **Тема:** Образование, студенты (книга, шапка выпускника, и т.д.)

Где найти иконки:

1. [Material Icons](#) — поиск "school", "person", "education"
2. [Flaticon](#) — поиск "student", "university"
3. [Icons8](#) — категория Education

Добавление иконки:

1. Скачайте иконку
2. Если это **PNG** — перетащите в **res/drawable/**
3. Если это **SVG** — конвертируйте в Vector Drawable:
 - Кликните правой кнопкой на **res** → **New** → **Vector Asset**
 - **Local file (SVG, PSD)** → выберите файл
 - Нажмите **Next** → **Finish**
4. Переименуйте в **ic_student_logo.png**



Вариант 2: Создание простой иконки вручную

Если у вас проблемы со скачиванием, создайте простую векторную иконку.

Создайте файл **res/drawable/ic_student_logo.xml**:

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="64dp"
    android:height="64dp"
    android:viewportWidth="64"
    android:viewportHeight="64">
    <!-- Простой круг с инициалами -->
    <path
        android:fillColor="#006C4C"
        android:pathData="M32,32m-28,0a28,28 0,1 1,56 0a28,28 0,1 1,-56 0"/>
    <!-- Буква S (Student) -->
    <path
        android:fillColor="#FFFFFF"
        android:pathData="M32,20 L32,20 Q40,20 40,26 Q40,32 32,32 Q24,32 24,38 Q24,44 32,44 L32,44"
        android:strokeWidth="4"
        android:strokeColor="#FFFFFF"
        android:fillAlpha="0"/>
</vector>
```

Это создаст зелёный круг с белой буквой "S" (Student).

10.2. Интеграция Scaffold в приложение

Теперь обернём наш список студентов в Scaffold, чтобы добавить верхнюю панель.

Откройте **MainActivity.kt** и найдите функцию **StudentApp()**.

Обновите функцию:

```

@Composable
fun StudentApp() {
    Scaffold(
        topBar = {
            StudentTopAppBar()
        }
    ) { it ->
        LazyColumn(contentPadding = it) {

```

Что мы сделали:

1. Добавили topBar:

- `topBar = { StudentTopAppBar() }` — слот для верхней панели
- Вызываем функцию `StudentTopAppBar()` (создадим далее)

2. Передали contentPadding:

- `{ it -> }` — лямбда получает `PaddingValues`
- `LazyColumn(contentPadding = it)` — применяет отступы
- Это нужно, чтобы контент не перекрывался с `TopAppBar`

Что такое contentPadding?

- **Scaffold** автоматически вычисляет отступы для **TopAppBar**
- Передаёт их через параметр **it** (**PaddingValues**)
- **LazyColumn** применяет эти отступы, чтобы контент начинался **под** панелью

10.3. Создание функции StudentTopAppBar

Теперь создадим саму верхнюю панель.

Добавьте новую Composable-функцию:

```

@Composable
fun StudentTopAppBar(modifier: Modifier = Modifier) {
    CenterAlignedTopAppBar(
        modifier = modifier
    )
}

```

Что мы сделали:

- Создали функцию `StudentTopAppBar()`
- Используем `CenterAlignedTopAppBar` — панель с центрированным заголовком
- Передаём **modifier** для гибкости

Добавьте импорт:

```
import androidx.compose.material3.CenterAlignedTopAppBar
```

На данном этапе панель пустая. Добавим содержимое!

10.4. Добавление заголовка с иконкой и текстом

Обновите функцию `StudentTopAppBar()`:

```
@Composable
fun StudentTopAppBar(modifier: Modifier = Modifier) {
    CenterAlignedTopAppBar(
        title = {
            Row(
                verticalAlignment = Alignment.CenterVertically
            ) {
                Image(
                    modifier = Modifier
                        .size(dimensionResource(R.dimen.image_size))
                        .padding(dimensionResource(R.dimen.padding_small)),
                    painter = painterResource(R.drawable.ic_student_logo),
                    contentDescription = null
                )
                Text(
                    text = stringResource(R.string.app_name),
                    style = MaterialTheme.typography.displayLarge
                )
            }
        },
        modifier = modifier
    )
}
```

Разберём пошагово:

1. Параметр `title`

`title` — это слот для содержимого заголовка. Мы передаём туда `Row` с иконкой и текстом.

2. `Row` с вертикальным выравниванием

- `Row` размещает элементы горизонтально
- `verticalAlignment = Alignment.CenterVertically` — выравнивает по центру вертикально
- Это нужно, чтобы иконка и текст были на одной линии

3. Иконка (`Image`)

Объяснение:

- `size()` — устанавливает размер 64dp (из `dimens.xml`)
- `padding()` — отступ 8dp
- `painterResource()` — загружает иконку `ic_student_logo`
- `contentDescription = null` — иконка декоративная, не несёт семантической информации для пользователей с нарушением зрения

4. Текст названия приложения

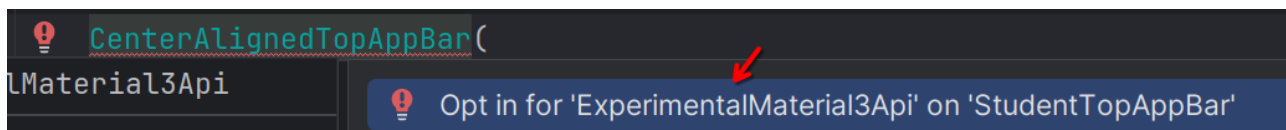
Объяснение:

- `stringResource(R.string.app_name)` — берёт название из `strings.xml` ("Student List")
- `style = MaterialTheme.typography.displayLarge` — применяет стиль **Abril Fatface, 36sp**
- Название короткое и важное — подходит стиль `displayLarge`

При необходимости добавьте импорты:

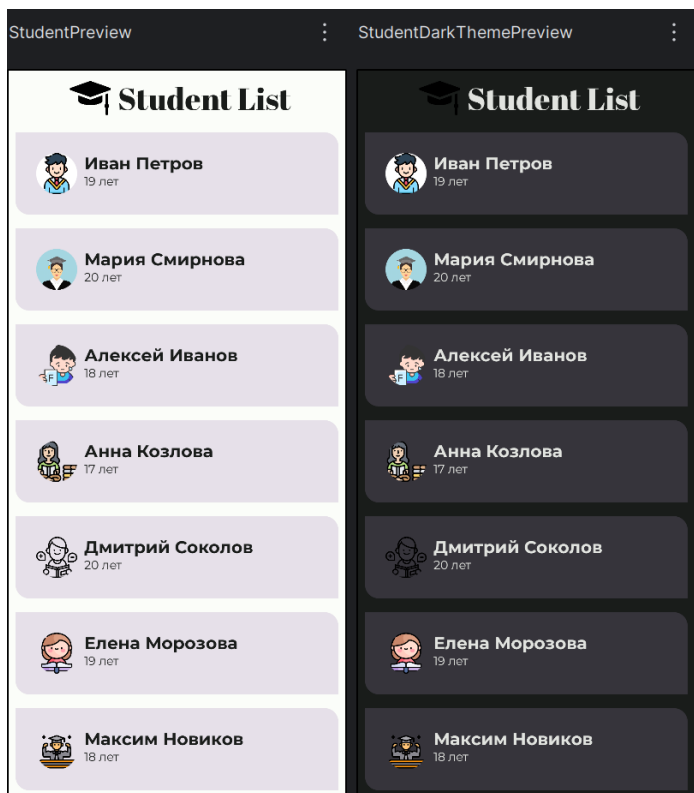
`import androidx.compose.ui.Alignment`

Также добавьте `ExperimentalMaterial3Api`:



10.5. Проверка в Preview

Откройте `StudentPreview()` и посмотрите на результат:



- Сверху появилась **верхняя панель**
- Иконка логотипа **слева от названия**
- Название приложения **крупным декоративным шрифтом**
- Элементы **выровнены по центру вертикально**
- Список студентов **начинается под панелью**

10.6. Настройка цвета TopAppBar (опционально)

По умолчанию **CenterAlignedTopAppBar** использует цвет **surface** из темы. Если хотите изменить цвет:

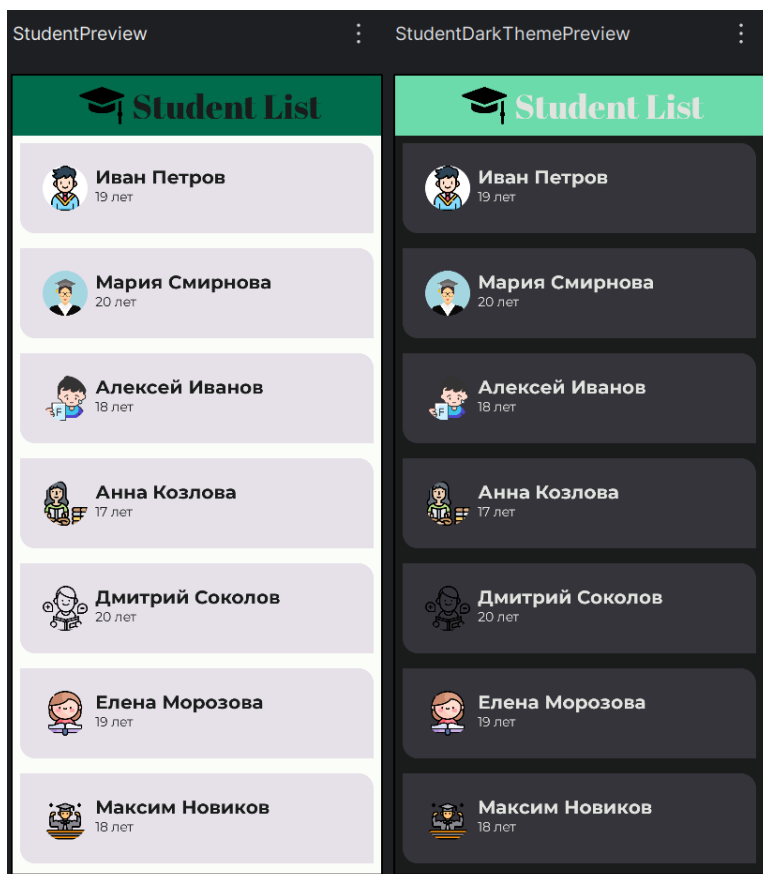
Добавьте параметр **colors**:

```
CenterAlignedTopAppBar(
    title = {...},
    colors = TopAppBarDefaults.centerAlignedTopAppBarColors(
        containerColor = MaterialTheme.colorScheme.primary
    ),
    modifier = modifier
```

При необходимости добавьте импорты:

`import androidx.compose.material3.TopAppBarDefaults`

Это сделает панель **зелёной** (цвет **primary** из темы).

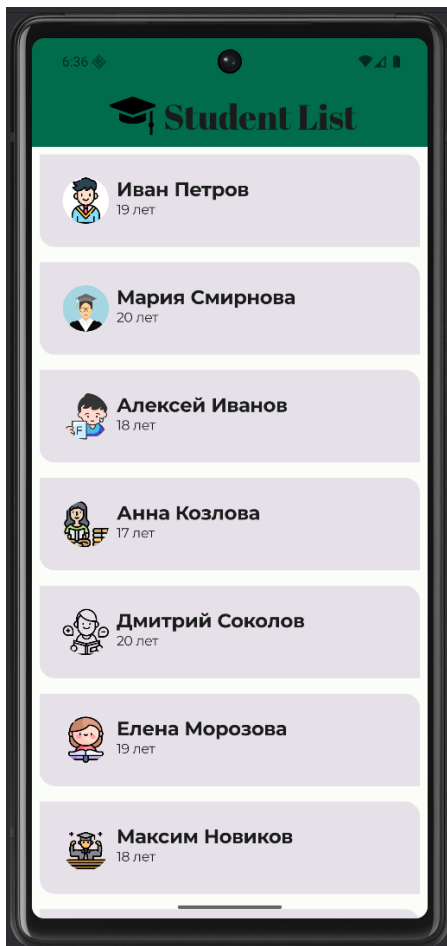


10.7. Запуск приложения

1. Нажмите **Run**

2. Проверьте приложение:

- Верхняя панель отображается
- Иконка и название выровнены
- Список начинается под панелью
- Нет наложений элементов



10.8. Просмотр в тёмной теме

Проверьте, как выглядит приложение в тёмной теме!

В Preview:

Прокрутите до `StudentDarkThemePreview()` — панель автоматически адаптировалась к тёмной теме.

На устройстве:

1. Откройте **Настройки** → **Оформление**
2. Включите **Тёмную тему**
3. Откройте приложение

GitHub

1. Сделайте **два** скриншота (**светлая** и **тёмная** тема):
2. Сохраните скриншоты в папку `/app/img/`
3. Назовите файлы:

`step10_appBar_lightTheme_FIO.png`

`step10_appBar_darkTheme_FIO.png`

Commit изменений:

`git add .`

`git commit -m "Step 10: Top App Bar added"`

`git push`

Шаг 11. Анимация

Анимация делает приложение более **интерактивным, интересным и понятным** для пользователей. Анимирование отдельных изменений на экране помогает пользователю увидеть, что именно изменилось.

Теория: Типы анимаций

Существует множество типов анимаций для пользовательского интерфейса:

- **Fade in/out** — элементы плавно появляются и исчезают
- **Slide** — элементы двигаются на экран или за его пределы
- **Transform** — элементы трансформируются интересными способами
- **Expand/Collapse** — элементы раскрываются и сворачиваются

Анимации помогают:

- Сделать интерфейс более **элегантным**
- Улучшить **пользовательский опыт**
- Показать **изменения состояния** элементов
- Привлечь **внимание** к важным элементам

11.1. Добавление иконки "Развернуть"

Сначала добавим иконку **Expand More (▼)** и **Expand Less (▲)** для раскрытия/сворачивания информации о студенте.

Теория: Иконки в Material Design

Иконки — это символы, которые помогают пользователям понять интерфейс, **визуально передавая** функцию элемента. Они часто вдохновлены объектами из физического мира.

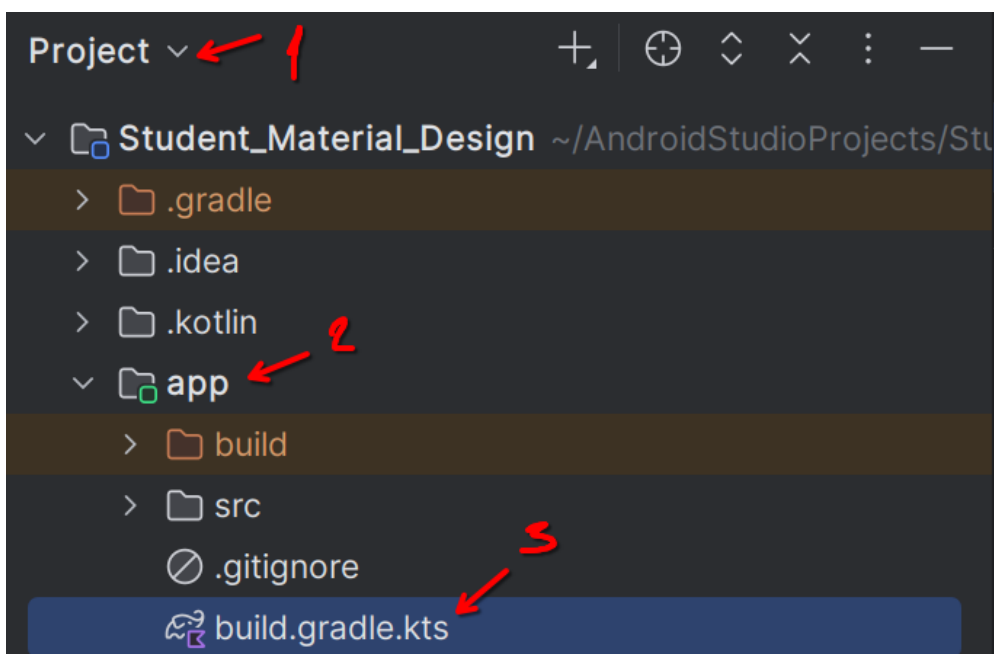
Пример: Карандаш в реальном мире используется для письма → его иконка означает "создать" или "редактировать".

Material Design предоставляет **множество иконок**, организованных по категориям.

Добавление зависимости Gradle

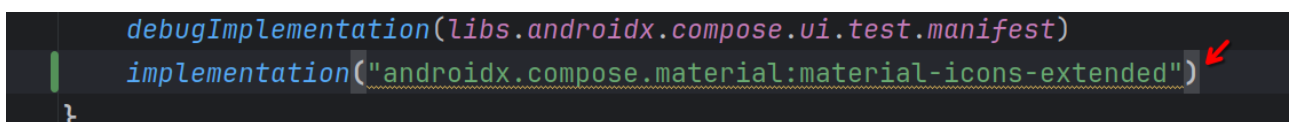
Добавим библиотеку **material-icons-extended** для использования расширенного набора иконок.

1. В панели **Project** откройте **Gradle Scripts** → **build.gradle.kts (Module :app)**

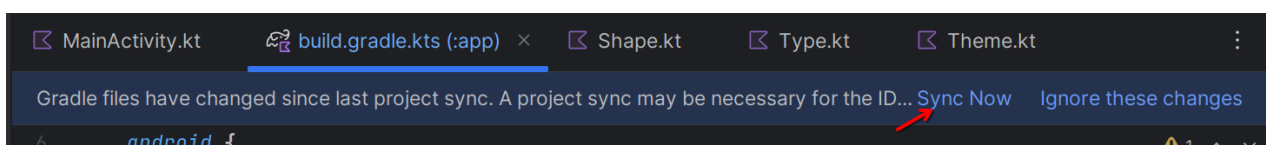


2. Прокрутите до конца файла и в блоке **dependencies {}** добавьте строку:

implementation("androidx.compose.material:material-icons-extended")



3. Нажмите **Sync Now** в появившемся окне (синхронизация Gradle)



Что мы делаем:

- Добавляем библиотеку с дополнительными иконками
- **material-icons-extended** содержит иконки **ExpandMore** и **ExpandLess**

- Gradle автоматически скачает и подключит библиотеку

11.2. Создание кнопки с иконкой

В **MainActivity** создадим **Composable**-функцию для кнопки раскрытия/сворачивания.

Добавьте новую функцию после **StudentItem()**:

```
@Composable
private fun StudentItemButton(
    expanded: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    IconButton(
        onClick = onClick,
        modifier = modifier
    ) {
        Icon(
            imageVector = if (expanded) Icons.Filled.ExpandLess else Icons.Filled
                .ExpandMore,
            contentDescription = stringResource(R.string
                .expand_button_content_description),
            tint = MaterialTheme.colorScheme.secondary
        )
    }
}
```

Параметры функции:

- **expanded: Boolean** — состояние (развёрнуто/свёрнуто)
- **onClick: () -> Unit** — лямбда-функция, вызываемая при клике
- **modifier: Modifier** — опциональный модификатор

IconButton:

- Создаёт кликабельную иконку
- **onClick** — обработчик клика

Icon:

- **imageVector** — выбор иконки в зависимости от состояния:
 - Если **expanded = true** → **ExpandLess** (▲)
 - Если **expanded = false** → **ExpandMore** (▼)
- **contentDescription** — описание для accessibility
- **tint** — цвет иконки (вторичный цвет темы)

Добавьте в strings.xml внутрь resources:

`<string name="expand_button_content_description">Развернуть или свернуть</string>`

Необходимые импорты:

```
import androidx.compose.material.icons.filled.ExpandLess
import androidx.compose.material.icons.filled.ExpandMore
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
```

11.3. Добавление состояния expanded

Теперь добавим переменную для отслеживания состояния раскрытия карточки.

В функции `StudentItem()`, в самом начале, добавьте:

```
@Composable
fun StudentItem(
    student: Student,
    modifier: Modifier = Modifier
) {
    + var expanded by remember { mutableStateOf(false) }
    Card(modifier = modifier) {
```

Что мы делаем:

`mutableStateOf(false)`:

- Создаёт наблюдаемое состояние со значением `false`
- При изменении значения Compose перерисует UI

`remember {}`:

- Сохраняет значение между рекомпозициями
- Без `remember` значение сбросится при каждой перерисовке

`by` делегат:

- Позволяет использовать `expanded` как обычную переменную
- Вместо `expanded.value = true` пишем просто `expanded = true`

Необходимые импорты:

```
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
```

11.4. Добавление кнопки в карточку

Добавим кнопку в конец строки с информацией о студенте.

В функции `StudentItem()`, в `Row` после `StudentInformation()`, добавьте:

```
StudentIcon(student.imageResourceId)
StudentInformation(student.name, student.age)
Spacer(modifier = Modifier.weight(1f))
StudentItemButton(
    expanded = expanded,
    onClick = { expanded = !expanded }
)
```

Что мы добавили:

Spacer(modifier = Modifier.weight(1f)):

- Заполняет **оставшееся пространство** в строке
- `weight(1f)` означает, что `Spacer` займёт всё свободное место
- Это "прижимает" кнопку к **правому краю**

Теория `Modifier.weight()`: `weight()` устанавливает ширину/высоту элемента пропорционально его весу относительно других взвешенных элементов.

В нашем случае **только `Spacer`** имеет вес, поэтому он занимает **всё оставшееся место**.

StudentItemButton:

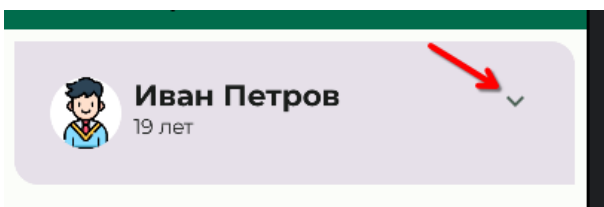
- `expanded = expanded` — передаём текущее состояние
- `onClick = { expanded = !expanded }` — инвертируем состояние при клике
 - Если `expanded = true` → станет `false`
 - Если `expanded = false` → станет `true`

Необходимые импорты:

`import androidx.compose.foundation.layout.Spacer`

Проверьте Preview:

Кнопка с иконкой ▼ должна появиться **справа** в каждой карточке.



11.5. Добавление описания студента

Создадим Composable для отображения **описания** студента (которое будет раскрываться/сворачиваться).

Создайте новую функцию:

```
@Composable
fun StudentDescription(
    @StringRes studentDescription: Int,
    modifier: Modifier = Modifier
) {
    Column(
        modifier = modifier
    ) {
        Text(
            text = stringResource(R.string.about),
            style = MaterialTheme.typography.labelSmall
        )
        Text(
            text = stringResource(studentDescription),
            style = MaterialTheme.typography.bodyLarge
        )
    }
}
```

Добавьте в strings.xml:

```
<string name="about">О студенте</string>
```

Объяснение кода

Column:

- Размещает элементы **вертикально**

Первый Text (заголовок):

- Текст "О студенте"
- Стиль `labelSmall` — маленький, жирный Montserrat (14sp)

Второй Text (описание):

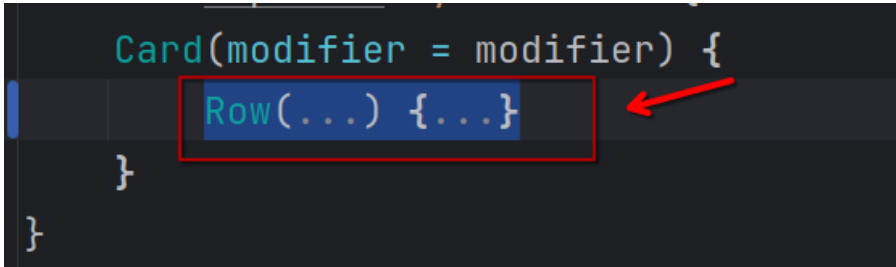
- Юмористическое описание студента
- Стиль `bodyLarge` — обычный Montserrat (14sp)

11.6. Интеграция описания в карточку

Теперь добавим описание в карточку студента, чтобы оно **раскрывалось** при **клике**.

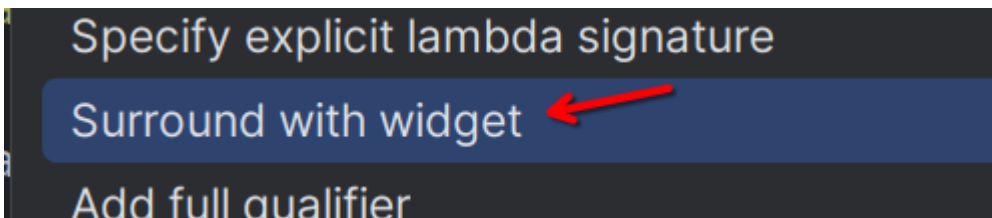
В функции `StudentItem()`, оберните **Row** в **Column**.

1. Выделяем Row:



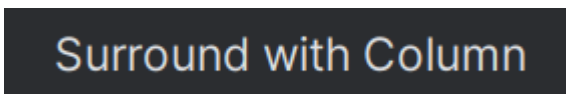
```
Card(modifier = modifier) {  
    Row(...) {...}  
}
```

2. Нажимаем **Alt+Enter** и выбираем **Surround with widget**:



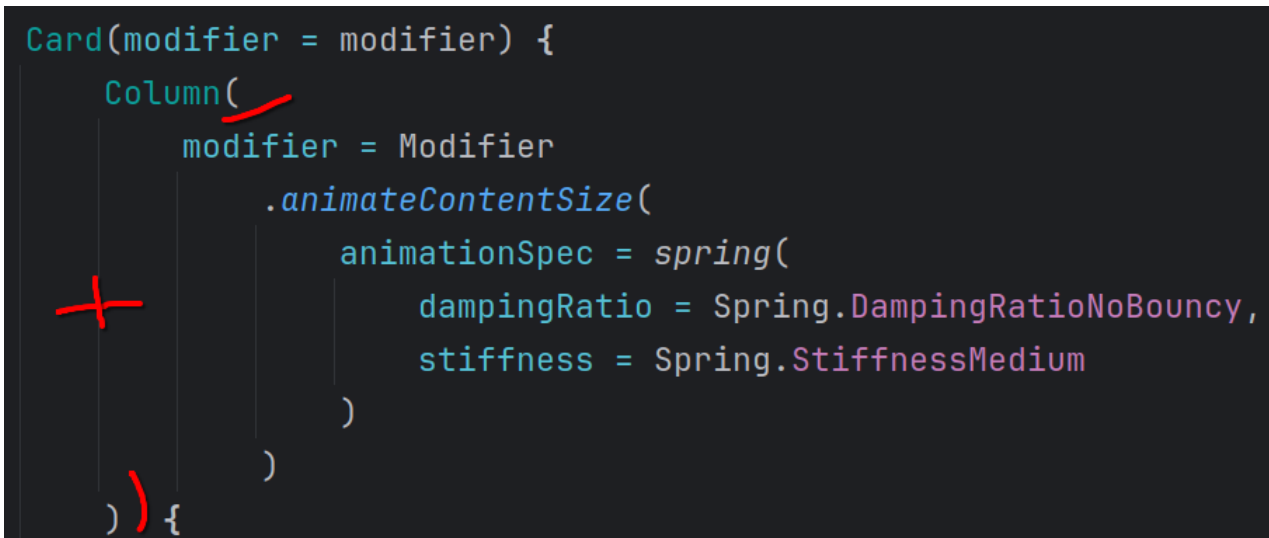
Specify explicit lambda signature
Surround with widget
Add full qualifier

Surround with Column:



Surround with Column

3. Для **Column** добавляем:



```
Card(modifier = modifier) {  
    Column(  
        modifier = Modifier  
            .animateContentSize(  
                animationSpec = spring(  
                    dampingRatio = Spring.DampingRatioNoBouncy,  
                    stiffness = Spring.StiffnessMedium  
                )  
            )  
    ) {  
        ...  
    }  
}
```

Соглашаемся на импорты

4. После **Row** добавим проверку:

```

) {
    Row(...) {...}
    if (expanded) {
        StudentDescription(
            student.description,
            modifier = Modifier.padding(
                start = dimensionResource(R.dimen.padding_medium),
                top = dimensionResource(R.dimen.padding_small),
                end = dimensionResource(R.dimen.padding_medium),
                bottom = dimensionResource(R.dimen.padding_medium)
            )
        )
    }
}

```

Итоговая структура функции **StudentItem()**:

```

@Composable
fun StudentItem(student: Student, modifier: Modifier = Modifier) {
    var expanded by remember { mutableStateOf(false) }
    Card(modifier = modifier) {
        Column(...) {
            Row(...) {...}
            if (expanded) {
                StudentDescription(...)
            }
        }
    }
}

```

Что мы сделали:

1. Обернули в **Column**:

- Теперь можем разместить **Row** и описание **вертикально**

2. Добавили условие **if (expanded)**:

- Описание отображается **только** если **expanded = true**
- При клике на кнопку состояние меняется, и описание появляется/исчезает

3. Добавили **animateContentSize()**:

- Анимировать изменение размера **Column**
- При раскрытии карточка **плавно увеличивается**
- При сворачивании **плавно уменьшается**

11.7. Теория: Spring Animation (Пружинная анимация)

Spring Animation — это **физическая анимация**, управляемая силой пружины.

Представьте, что вы перетаскиваете иконку приложения по экрану и отпускаете её. Иконка "**прыгает**" **обратно** на исходное место, как будто к ней прикреплена невидимая пружина.

Параметры пружины:

1. Damping Ratio (Коэффициент затухания):

- Определяет "**упругость**" пружины
- **Высокий** → нет отскока, плавное движение
- **Низкий** → сильный отскок, пружинистое движение

Значения:

- `DampingRatioNoBouncy` — без отскока (плавно)
- `DampingRatioLowBouncy` — слабый отскок
- `DampingRatioMediumBouncy` — средний отскок
- `DampingRatioHighBouncy` — сильный отскок

2. Stiffness (Жёсткость):

- Определяет, **как быстро** пружина движется к конечной точке
- **Высокая** → быстрое движение
- **Низкая** → медленное движение

Значения:

- `StiffnessVeryLow` — очень медленно
- `StiffnessLow` — медленно
- `StiffnessMedium` — средняя скорость
- `StiffnessHigh` — быстро

В нашем приложении:

```
animationSpec = spring(  
    dampingRatio = Spring.DampingRatioNoBouncy, // Без отскока  
    stiffness = Spring.StiffnessMedium           // Средняя скорость  
)
```

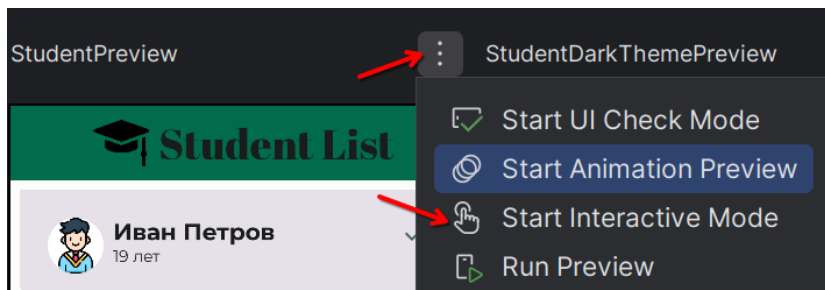
Карточка **плавно** раскрывается/сворачивается **без пружинистого эффекта**.

11.8. Тестирование анимации в Interactive Mode

Android Studio позволяет **взаимодействовать** с Preview без запуска приложения.

P.S. Лично у меня были проблемы с отображением, но возможно в учебной аудитории на ОС Windows всё отобразится корректно.

1. Откройте **Design** панель с StudentPreview
2. Наведите мышь на **StudentPreview** в Design Pane
3. Нажмите кнопку **Interactive Mode** в правом верхнем углу

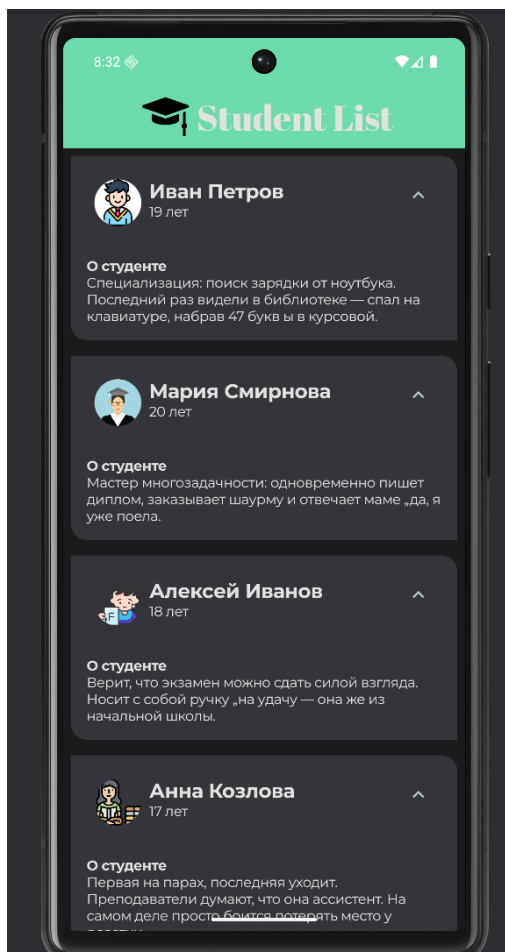


4. Кликните на иконку ▼ в любой карточке
5. Описание студента плавно раскроется ⬇️
6. Иконка изменится на ▲
7. Кликните снова — описание плавно свернётся ⬆️

Для выхода из режима: Нажмите **Stop Interactive Mode** в левом верхнем углу.

11.9. Запуск приложения

1. Нажмите **Run**



2. Проверьте на устройстве:

- Кнопка ▼ справа в каждой карточке
- При клике карточка **плавно раскрывается**
- Показывается описание студента
- Иконка меняется на ▲
- При повторном клике карточка **плавно сворачивается**

GitHub

1. Сделайте два скриншота (**развёрнутой** и **свёрнутой** карточки)

2. Сохраните скриншоты в папку `/app/img/`

3. Назовите файлы:

`step11_collapsed_state_lightTheme_FIO.png`

`step11_expanded_state_lightTheme_FIO.png`

Commit изменений:

git add .

git commit -m "Step 11: Animation and expand/collapse functionality added"

git push

Самостоятельное задание: Оформление README.md

Создайте файл **README.md** в корне проекта, который будет описывать вашу лабораторную работу.

Требования к README.md:

Файл должен содержать следующие разделы (напишите их самостоятельно):

1. **Название проекта** — название и краткое описание приложения
2. **Цель работы** — что вы изучали в этой лабораторной
3. **Функциональность** — список реализованных возможностей приложения (что умеет делать приложение)
4. **Технологии** — какие технологии и библиотеки использовались (Jetpack Compose, Material Design 3, и т.д.)
5. **Автор** — ваше ФИО и группа

После создания **README.md**:

git add README.md

git commit -m "Add README documentation"

git push